Beyond Rules: How Large Language Models are Redefining Cryptographic Misuse Detection

by

Zohaib Masood

A thesis submitted to the
School of Graduate and Postdoctoral Studies in partial
fulfillment of the requirements for the degree of

Master of Science in Computer Science

Faculty of Science
University of Ontario Institute of Technology (Ontario Tech University)
Oshawa, Ontario, Canada
March 2025

© Zohaib Masood, 2025

THESIS EXAMINATION INFORMATION

Submitted by: **Zohaib Masood**

Master of Science in Computer Science

Thesis title: Beyond Rules: How Large Language Models are Redefining Cryptographic Misuse Detection

An oral defense of this thesis took place on March 13, 2025 in front of the following examining committee:

Examining Committee:

Chair of Examining Committee	Dr. Christiano Politowski
Research Supervisor	Dr. Miguel Vargas Martin
Examining Committee Member	Dr. Patrick Hung
Thesis Examiner	Dr. Shahram S. Heydari, Ontario Tech University

The above committee determined that the thesis is acceptable in form and content and that a satisfactory knowledge of the field covered by the thesis was demonstrated by the candidate during an oral examination. A signed copy of the Certificate of Approval is available from the School of Graduate and Postdoctoral Studies.

Abstract

The use of Large Language Models (LLMs) in software development is rapidly growing, with developers increasingly relying on these models for coding assistance, including security-critical tasks. Our work presents a comprehensive comparison between traditional static analysis tools for cryptographic API misuse detection—CryptoGuard, CogniCrypt, and Snyk Code—and the LLMs—GPT, Llama, Claude, and Gemini. Using benchmark datasets (OWASP, CryptoAPI, and MASC), we evaluate the effectiveness of each tool in identifying cryptographic misuses. Our findings show that GPT 4-o-mini surpasses current state-of-the-art static analysis tools on the CryptoAPI and MASC datasets, though it lags on the OWASP dataset. Additionally, we assess the quality of LLM responses to determine which models provide actionable and accurate advice, giving developers insights into their practical utility for secure coding. This study highlights the comparative strengths and limitations of static analysis versus LLM-driven approaches, offering valuable insights into the evolving role of AI in advancing software security practices.

Keywords: Cryptographic Misuse Detection; Large Language Models; Static Analysis Tools; Software Security

Author's Declaration

I hereby declare that this thesis consists of original work of which I have authored. This is a true copy of the thesis, including any required final revisions, as accepted by my examiners. I authorize the Ontario Tech University to lend this thesis to other institutions or individuals for the purpose of scholarly research. I further authorize the Ontario Tech University to reproduce this thesis by photocopying or by other means, in total or in part, at the request of other institutions or individuals for the purpose of scholarly research. I understand that my thesis will be made electronically available to the public.

Zohaib Masood

Statement of Contributions

I hereby certify that I am the sole author of this thesis and that no part of this thesis has been published. I have used standard referencing practices to acknowledge ideas, research techniques, or other materials that belong to others. Furthermore, I hereby certify that I am sole source of the creative works and/or inventive knowledge described in this thesis.

Dedication

I dedicate this work to my family, especially my father, Masood Ashraf, whose unwavering support and encouragement have been my strength throughout this journey. Their belief in me has kept me going through every challenge. I also would like to thank my mentors and peers, whose guidance and insights have enriched my research and shaped my perspective.

Acknowledgements

This research was supported by the Natural Sciences and Engineering Research Council of Canada, Grant # DDG-2024-00031. We also thank the Vulnerability Research Centre at the Communications Security Establishment, Canada, for supporting this research.

Contents

Thesis Examination Information **ii**

Abstract **iii**

Author's Declaration **iv**

Statement of Contributions **v**

Dedication **vi**

Acknowledgment **vii**

1 Introduction **1**

2 Motivation and Related Work **6**

2.1 Motivation 6

2.2 Related Work 8

3 Methodology **15**

3.1 Selection Criteria 15

3.1.1 Static Tools 15

3.1.2 Benchmark Datasets 16

3.1.3 Large Language Models 18

3.2 Evaluation Metrics 19

3.2.1 LLM Specific Metrics . . . 19
3.3 Experiments . . . 22
3.3.1 LLM vs Static Tools . . . 22
3.3.2 Comparing LLMs . . . 23
3.3.3 Evaluating LLM Responses . . . 23

4 Results and Findings **24**
4.1 LLMs vs Static Tools . . . 24
4.1.1 CryptoAPI Benchmark . . . 24
4.1.2 OWASP Benchmark . . . 27
4.1.3 ROC Comparison . . . 29
4.1.4 Time Comparison . . . 30
4.2 Comparing LLMs with Datasets . . . 32
4.2.1 ROC Comparison . . . 33
4.3 Evaluating LLM Responses . . . 36

5 Discussion and Ecological Validity **40**
5.1 Discussion . . . 40
5.1.1 F-score vs Recall . . . 40
5.1.2 Strengths of Large Language Models . . . 41
5.1.3 Actionability vs. Specificity in LLM Responses . . . 41
5.1.4 Benchmark Inconsistencies . . . 43
5.1.5 LLM Hallucination . . . 43
5.2 Ecological Validity . . . 44
5.2.1 Applicability to Real World Scenarios . . . 44
5.2.2 Threats to Validity . . . 47

6 Future Work and Conclusion **49**
6.1 Future Work . . . 49

6.2 Conclusion 51

Bibliography **54**

A Appendix **61**

A.1 LLM Prompt 61

A.2 LLM Responses 63

List of Tables

2.1 Overview of Existing Work on LLM-Based Detection 13

3.1 Version of Static Analysis Tools 16

3.2 Labels of Benchmarking Datasets 18

3.3 Version of Large Language Models 19

3.4 Actionable Keywords 20

4.1 Evaluation Metrics for CryptoAPI and OWASP Benchmark 25

4.2 Execution Time of Tools for CryptoAPI and OWASP Benchmark 30

4.3 Comparison of LLMs on Different Benchmarks[1] 33

4.4 Actionability and Specificity of LLMs Across Benchmark Datasets 36

5.1 Comparison of F-1 and F-2 Scores for OWASP 41

A.21 LLM Responses to Test Cases from Different Datasets 66

List of Figures

3.1 Evaluation Methodology for Comparing Static Tools with LLMs 22

4.1 Tools Comparison on CryptoAPI Benchmark 25

4.2 Detection Results for CryptoAPI Benchmark Across Different Tools . . . 26

4.3 Tools Comparison on OWASP Benchmark 27

4.4 Detection Results for OWASP Benchmark Across Different Tools 28

4.5 TPR vs FPR for Static Tools and GPT 29

4.6 TPR vs FPR of LLMs for CryptoAPI 34

4.7 TPR vs FPR of LLMs for OWASP 34

4.8 TPR vs FPR of LLMs for MASC 35

4.9 GPT Actionable Keywords Occurrence 37

4.10 Gemini Actionable Keywords Occurrence 38

4.11 Llama Actionable Keywords Occurrence 38

4.12 Claude Actionable Keywords Occurrence 39

Chapter 1

Introduction

Protecting sensitive data on digital devices from eavesdropping or forgery relies primarily on cryptography. To ensure effectiveness in protection, the cryptographic algorithms employed must be conceptually secure, implemented accurately, and utilized securely within the relevant application. Despite the existence of mature and still secure cryptographic algorithms, numerous studies have pointed out that application developers face challenges in utilizing the Application Programming Interfaces (APIs) of libraries that incorporate these algorithms. As an illustration, Lazar et al. [25] examined 269 vulnerabilities related to cryptography and discovered that merely 17% are associated with flawed algorithm implementations, while the remaining 83% stem from the misuse of cryptographic APIs by application developers. Additional research indicates that around 90% of applications utilizing cryptographic APIs include at least one instance of misuse [5, 13].

Software designers and developers need to tackle this security flaw by ensuring that their applications encrypt the sensitive data they handle and store. Despite the availability of educational materials [20] aimed at raising awareness and providing guidance on secure code development, many developers remain unaware of security considerations [48]. Training initiatives are not universally received by programmers [48], and security is frequently treated as a secondary, desired goal [46], rather than a mandatory one, depending

on the perceived risk and criticality of the developed application. Typically, developers prioritize meeting functionality and time-to-market requirements as their primary goals.

In exploring the factors contributing to this prevalent misuse, researchers previously triangulated findings from four empirical studies, including a survey involving Java developers with prior experience using cryptographic APIs [31]. Their findings reveal that a significant majority of participants encountered difficulties in utilizing the respective APIs. Several other tools [22,23,37,49] have been developed to detect cryptographic misuses, however, the robustness of these tools has been still in question [3, 50]. In addition to it, novice developers have started adopting AI-assisted tools to code their programming problems. Recent advancements encompass Github's Copilot [19], DeepMind's AlphaCode [26], Amazon's Q Developer [2], Tabnine [44], Google's Gemini [18], Meta's Llama [28], Anthropic's Claude [7], Qwen's QwenLM [36], DeepSeek [12] and Open AI's ChatGPT [4]—ten systems capable of translating a problem description into code. Prior results suggest that Open AI's ChatGPT reliably produces Java programming solutions known for their elevated readability and well-organized structure [32].

Prior research has primarily tested Large Language Models (LLMs) using a limited set of benchmark datasets, providing an initial understanding of their performance. However, a comprehensive study evaluating the effectiveness of LLMs across a wider range of benchmarks is still lacking. Furthermore, no previous work has closely examined the quality of LLM responses, particularly with regard to the accuracy and actionability that would validate these responses for practical use. This study addresses these gaps by conducting an analysis of LLM performance across diverse datasets and introducing a framework for assessing response quality. In our work, we aim to assess the efficacy of identifying cryptographic misuses in the provided code and compare it with state-of-the-art (SOTA) static cryptographic analysis tools from literature such as CogniCrypt [23], CryptoGuard [37], and an industry-based tool, Snyk Code [42]. To evaluate LLMs accuracy against these tools, we utilized well-known benchmark datasets employed in previous

literature for cryptographic misuse detection, namely OWASP Bench [34], CryptoAPI Bench [1]. In addition, we investigated LLMs robustness in detecting mutated test cases, an area where many static tools often falter [3]. Furthermore, we evaluated the quality of responses provided by LLMs using two metrics, Actionability and Specificity, which assess whether the response can assist developers in identifying the cause of misuse and in implementing a fix.

RQ1. How effective are LLMs in detecting cryptographic misuses compared to other static tools?

The static tools tested focus on slightly different pattern sets, leading to varied trade-offs in precision and recall, both among the tools themselves and in comparison to LLMs. To evaluate the effectiveness of LLMs in detecting cryptographic misuses compared to static tools, we ran test cases from the CryptoAPI and OWASP benchmarks. Based on the findings, GPT had a better detection rate across both benchmarks. For CryptoAPI, GPT missed only 3 true instances, with CryptoGuard lagging behind at 24 misses. In the OWASP benchmark, GPT identified all true instances, while CryptoGuard missed 40 true misuse cases. However, for the OWASP benchmark, GPT had a high false positive rate, indicating that no tool is universally superior across both benchmarks.

RQ2. How robust are LLMs in detecting mutated test cases that other static tools fail to detect?

To evaluate the robustness of LLMs, we ran test cases from a manually curated MASC dataset [3] against LLMs to see if LLMs can detect these mutations of cryptographic code effectively. Our results suggest that GPT performed better than other LLMs in detecting cryptographic mutations.

RQ3. How do prevalent LLMs compare in detecting cryptographic misuse and providing actionable, specific guidance for developers?

To address this, we compared the performance of LLMs across both benchmarks and a mutated dataset to evaluate which LLM performs best. Additionally, we introduced a method to assess whether an LLM's response can assist developers in fixing misuses. Since LLMs often generate text-heavy responses that may not effectively help developers, we implemented a keyword-based approach to analyze LLM outputs. This approach identifies which LLM provides more actionable and specific guidance for developers to address misuse instances.

Our main contributions from this work are as follows:

- We are the first to conduct a comprehensive evaluation of the effectiveness of LLMs in detecting cryptographic misuses, comparing their performance to that of SOTA static analysis tools. It highlights the strengths and weaknesses of LLMs compared to static tools, offering insights into their reliability and applicability for secure software development.

- We assess the robustness of LLMs by evaluating their ability to detect cryptographic misuse in mutated test cases that static tools often miss. This contribution explores whether LLMs can adapt to variations in misuse patterns, providing an understanding of their resilience and potential superiority in handling unconventional or complex misuse scenarios.

- To systematically assess the practicality of LLM-generated guidance, we develop a keyword-based framework that scans LLM responses for actionable elements. This framework serves as a tool to measure the usability of LLM outputs for developers, supporting the identification of models that best fulfill developers' needs in addressing cryptographic misuse. To the best of our knowledge, this is the first work that explores the quality of LLM responses.

In the next chapter, we cover the motivation and related work in this domain. Chapter

3 explains the methodology, while Chapter 4 presents the results and findings. Chapter 5 covers the discussion, followed by ecological validity and limitations. Lastly, Chapter 6 discusses possible future directions and provides a conclusion.

Chapter 2

Motivation and Related Work

2.1 Motivation

To proactively identify vulnerabilities before software release, individuals without specialized expertise—such as software developers or quality assurance teams—are likely to integrate cryptographic API misuse detectors, commonly known as crypto-detectors, into the Continuous Integration/Continuous Delivery (CI/CD) pipeline. Examples of these detectors include CryptoGuard [8] and CogniCrypt [23]. Consequently, the incapability of a crypto detector to identify an instance of misuse that it claims to detect has a direct impact on the security of end-user software. We illustrate this challenge with an example.

Imagine Johnny, a Java developer utilizing CryptoGuard [8], an advanced crypto detector, to pinpoint cryptographic vulnerabilities in his software before its release. In one of his applications, Johnny opts to employ the DES cipher, outlined as follows:

Cipher cipher = Cipher . getInstance (“des”);

This signifies an instance of misuse involving the use of the vulnerable DES cipher. CryptoGuard falls short in recognizing this vulnerability because Johnny utilizes “des”

instead of "DES" as the parameter [3]. However, this presents a concern because the difference in lower case for the parameter lacks functional significance, as Java officially supports both upper case and lower case. Due to CryptoGuard's oversight of this vulnerability, Johnny might erroneously believe that his application is secure and may proceed with releasing it to end-users.

The same example was tested with ChatGPT by providing a starting prompt "Can you detect cryptographic API misuse in the given Java code?". ChatGPT generated the following response:

"The line of code you provided initializes a Cipher object in Java using the DES (Data Encryption Standard) algorithm. However, it's important to note that DES is considered insecure for many applications due to its small key size (56 bits) and vulnerability to brute-force attacks. It is generally recommended to use more secure algorithms like AES (Advanced Encryption Standard) instead."

In other words, these static tools for cryptographic misuse detection could have fundamental flaws preventing them from detecting even straightforward instances of crypto-API misuse found in applications. On the other hand, ChatGPT's accurate response to such test cases raises a curiosity to test whether LLMs can effectively detect cryptographic misuses better than these static analysis tools. To extend it, novice developers intrigued by the power of LLMs capabilities, have started adapting to the technology by using such platforms to look for coding solutions. With the increasing use of AI to look for code solutions by novice developers, little research is done in this area to explore whether prominent platforms, such as ChatGPT, are effective in detecting these cryptographic misuses. It will also likely conclude whether the code provided by these LLMs for cryptographic problems is secure or not. This insight guides our approach to systematically compare widely known crypto detectors with LLMs.

Misusing cryptographic APIs can have serious consequences, as demonstrated by vulnerabilities like Heartbleed. Heartbleed [9], a bug in OpenSSL discovered in 2014, allowed

attackers to access sensitive information such as passwords and private keys from servers, affecting millions of websites and services. This flaw highlighted how small mistakes in cryptographic API usage could lead to major security issues. Similar vulnerabilities, such as improper encryption handling [33] and incorrect key management [10], continue to threaten system security, potentially exposing private data and eroding trust in security systems. For example, the Microsoft Azure Key Vault [30] has faced security challenges, with vulnerabilities involving improper configurations in role-based access control (RBAC) and access policies [21]. These misconfigurations could allow attackers to escalate privileges and gain unauthorized access to sensitive secrets, exposing cryptographic keys and passwords. To mitigate such risks, it is essential to follow best practices for key management, including strict access controls, regular auditing, and ensuring proper configuration of roles and permissions. Microsoft continues to address these vulnerabilities and provides guidance to help users secure their Azure Key Vault instances.

Although early analysis showed promising results for Open AI's ChatGPT, an intensive study was conducted to conclude the effectiveness of LLMs in detecting cryptographic misuses. LLMs can generate and understand code in various programming languages, however, static tools and benchmarking datasets used for comparison in our study rely only on Java. Therefore, the scope of our work is limited to Java as a programming language.

2.2 Related Work

Recently, security researchers have expressed significant interest in externally validating static analysis tools [3, 50]. It highlights the weaknesses of existing static analysis tools in detecting cryptographic misuses. Ying et al. [50] highlighted the weaknesses of static tools and how effectively it allowed developers to fix those misuse instances. Amit et al. [3] introduced the MASC framework, which introduces mutations in the code and

demonstrates how static tools fail to identify common misuse instances. Specifically, there's a growing recognition that while static analysis security tools are theoretically sound, they can be "soundy" in practice. This means they consist of a core set of sound decisions but also include certain strategically unsound choices made for practical reasons, such as performance or precision considerations [8]. In this chapter, we provide a brief overview of static analysis tools, some of which will be used for comparison with LLMs. Additionally, we discuss similar work done in this domain so far.

To enhance cryptographic security, various tools have been developed to support developers in ensuring secure coding practices. Crypto-Assistant, introduced by Rodriguez et al. [40], is an Eclipse plugin that integrates with Hibernate Tools to help developers encrypt sensitive data. Aimed at developers without cryptography expertise, the tool simplifies encryption by leveraging Jasypt to encrypt database columns while persisting Java objects. It allows developers to choose encryption algorithms and provides security warnings, such as discouraging DES usage. However, developers must manually select which columns to encrypt, which can be a challenge. On the other hand, CRYLOGGER, introduced by Piccolboni et al. [35], takes a dynamic approach to detect cryptographic API misuse in Android apps. Unlike static tools, CRYLOGGER runs the app, collects log data, and analyzes it offline. It identifies rule violations related to parameter values, randomness, and value reuse. In a study of 1800 Android apps, CRYLOGGER uncovered security issues like broken hash algorithms and weak password protection, suggesting that dynamic tools complement static analysis by addressing vulnerabilities that only manifest during runtime.

While encryption tools and dynamic analysis play significant roles in secure coding, other approaches focus on guiding developers toward more secure solutions during their workflow. Fischer et al. [17] introduced a nudging system to guide developers towards secure coding practices within their workflow by using structure2vec to embed a code graph and link related problems on Stack Overflow. This system nudges developers to-

ward secure solutions, and a user study revealed that the nudge group, with access to these prompts, outperformed the control group in generating secure code. Similarly, Fischer et al. [16] enhanced developer resources like Stack Overflow, Google, and GitHub by re-ranking search results to prioritize secure solutions. Their study found that 22.8% of the top Google search results linked to insecure code, and a subsequent study showed that the treatment group, which received secure results, was more likely to submit secure solutions. Additionally, FluentCrypto [22] offers a Node.js crypto API wrapper that prevents misuse by using CryRule, a domain-specific language that defines constraints for crypto objects. This approach allows developers to configure a secure API without dealing with low-level complexities, and the tool issues warnings when the developers' code violates the defined rules. A study with eight developers demonstrated that FluentCrypto significantly improved the creation of secure solutions compared to using the standard Node.js API.

In contrast to tools that focus on guiding developers or enhancing developer resources, other approaches address vulnerabilities through advanced misuse detection methods and domain-specific applications. Ren et al. [39] introduced a novel approach for detecting API misuses by examining API caveat knowledge instead of relying on usage patterns. They created a unique API-constraint knowledge graph using reference documentation, capturing relations such as call-order, condition-checking, and exception handling. This approach enables the detection of misuses such as missing calls, condition checks, and exception handling, outperforming traditional pattern-based detectors. Similarly, CryptoREX [49] addresses security concerns in IoT devices by automating the analysis of cryptographic misuse in IoT firmware. By using a unified intermediate representation (IR), CryptoREX standardizes the analysis across various firmware architectures. The tool applies static data-flow analysis and uses Valgrind's VEX IR and PyVEX for processing. An analysis of 1327 firmware images revealed 679 crypto-misuse bugs, pointing out issues like the use of ECB mode encryption and constant encryption parameters. This

highlights the need for better security practices in IoT firmware development, providing valuable insights for enhancing cryptographic implementations in this space.

Among the various approaches to detecting cryptographic misuses, static analysis tools have demonstrated the highest accuracy in identifying vulnerabilities by systematically analyzing code without execution. *CryptoGuard* [37, 43] expands on Soot [43], a widely employed program analysis framework, for the static analysis of Java bytecode [45]. It specifically addresses vulnerabilities outlined in the Common Weakness Enumeration (CWE) [11]. The CWE is a publicly maintained list of software and hardware weaknesses that can lead to security vulnerabilities. Each weakness is assigned a unique identifier to standardize classification and aid in vulnerability management. CryptoGuard specifically addresses vulnerabilities such as improper certificate validation (CWE-295), where a system fails to verify the legitimacy of security certificates, potentially allowing man-in-the-middle attacks; inadequate encryption strength (CWE-326), which results in weak cryptographic protection; and use of a broken or risky cryptographic algorithm (CWE-327), where outdated or insecure algorithms compromise data security. Additionally, it mitigates the use of insufficiently random values (CWE-330), a flaw that can weaken cryptographic keys or authentication mechanisms; selection of a less secure algorithm during negotiation (CWE-757), which can lead to algorithm downgrade attacks; and the use of hard-coded credentials (CWE-798), a critical issue that can allow unauthorized access if credentials are exposed. To attain high precision in vulnerability detection, CryptoGuard employs both backward and forward slicing in a context- and field-sensitive manner. Nevertheless, since eight of CryptoGuard's rules pertain to constant value usage, blindly applying existing slicing techniques may inaccurately flag constants covered by program slices but unrelated to security. To address this, CryptoGuard introduces refinement algorithms that eliminate false alarms based on cryptography domain knowledge.

CogniCrypt [23] assists developers in appropriately utilizing APIs through two pri-

mary methods. Firstly, for common tasks like data encryption, CogniCrypt generates code based on high-level task descriptions provided in English. Secondly, CogniCrypt employs rules specified in a domain-specific language (DSL) called CrySL [24] to identify API misuses. CogniCrypt translates CrySL rules into context-sensitive, flow-sensitive, and demand-driven static analysis, enabling users to enhance the tool's capabilities by creating new rules. In case of identified API misuse, CogniCrypt provides guidance on its resolution, such as substituting an insecure parameter value with a secure one. Its pattern set pertains to CWE-295, CWE-326, CWE-327, CWE-330, CWE-757, and CWE-798.

Snyk Code [42], is a tool developed by Snyk that focuses on static application security testing (SAST). It helps developers identify and fix vulnerabilities in their code. Snyk Code integrates with various development environments and continuously scans codebases for security issues, providing actionable insights and recommendations to remediate detected vulnerabilities. Snyk Code stands out because of its developer-friendly approach, offering integrations with popular IDEs, CI/CD pipelines, and repositories. It uses machine learning and semantic analysis to detect issues, ensuring high accuracy and relevance in its findings. The tool supports multiple programming languages and frameworks, making it versatile for various development needs.

Static analysis tools often suffer from a high rate of false positives, flagging issues that do not pose a threat and creating a disconnect between reported misuse alerts and real vulnerabilities. Chen et al. [6] examine these limitations by analyzing the rules, models, and implementations of such tools, highlighting the need for improvements in their precision and usability. In contrast, the use of LLMs for code analysis is gaining traction, as demonstrated by Fang et al. [14], who concluded that advanced LLMs like GPT 3.5 and 4.0 offer promising capabilities for more accurate code review. This suggests that LLMs might mitigate the issue of false positives prevalent in static tools, providing a potentially safer and more effective alternative for detecting vulnerabilities.

Liu et al. [27] highlight the promising potential of using ChatGPT for vulnerability

Name	Benchmarks			LLM-based Detection	Evaluating Quality of LLM Response
	CryptoAPI	MASC	OWASP		
Firouzi et al. [15]	✓	×	×	✓	×
Xia et al. [47]	✓	✓	×	✓	×
Our Work	✓	✓	✓	✓	✓

Table 2.1: Overview of Existing Work on LLM-Based Detection

management. Their research showed that ChatGPT is notably effective in tasks such as generating titles for software bug reports. Similarly, with the rise in popularity of LLMs, researchers have started exploring the domain of detecting cryptographic misuses using LLMs [15, 47]. However, no prior work has examined the extent to which the text-based responses of LLMs assist developers in fixing misuse instances.

Despite the progress made by static analysis tools in detecting cryptographic misuse, significant gaps remain in their coverage and adaptability to varied misuse patterns. Existing tools often lack robustness when faced with mutated misuse instances, limiting their effectiveness in real-world scenarios. Prior research [29] has analyzed the detection of cryptographic misuses using LLMs but have been limited in the models used for comparison. This study extends previous work by incorporating a broader selection of LLMs to enable a more comprehensive evaluation against static analysis tools as well as among LLMs themselves. Additionally, this work introduces ROC-annotated plots to facilitate a clearer comparison of model performance. Our study addressed these gaps by conducting a comprehensive evaluation of static tools against LLMs using the OWASP and CryptoAPI benchmarks. Additionally, we tested the robustness of LLMs with mutated misuse cases, assessing their adaptability beyond typical patterns. To further assist developers, we introduced a keyword-based approach to measure the actionability of LLM responses and specificity to evaluate whether LLMs provide relevant examples to aid in code correction. By addressing these critical aspects, our work explored the potential of replacing static tools with LLMs to assist developers in writing secure code. Table 2.1 provides an overview of existing work in LLM-based detection, highlighting the unique

aspects and contributions of our work compared to prior research.

Chapter 3

Methodology

This chapter describes our methodology for evaluating LLMs in cryptographic misuse detection. We assessed LLMs against various datasets and further analyzed their responses to determine their effectiveness in detecting misuse instances.

3.1 Selection Criteria

3.1.1 Static Tools

A range of tools exists in both literature and industry for detecting cryptographic vulnerabilities in Java code, as most prior research has primarily focused on this language. We shortlisted two tools from academia and one from industry. The two academic tools selected were based on their detection superiority compared to other existing tools [1, 3, 37, 50]. For academia, we focused on tools that accept input in the form of a JAR file and are generic rather than specific to a particular framework. As a result, tools specific to Android applications were excluded from this study. We also prioritized tools that have gained popularity in the literature to enhance the credibility of our work when comparing them with LLM models. We selected Cryptoguard [37] and CogniCrypt SAST [23] as our static tools from academia. Both Cryptoguard and CogniCrypt were

set up separately on an Ubuntu VM to run test cases through the terminal. Cryptoguard requires Java 8, while CogniCrypt requires at least Java 11 for setup. For the industry-related tool, we chose Snyk Code [42], a popular option for comparison with other static tools and LLMs. Snyk Code was selected for comparison due to its targeted focus on security vulnerabilities, including cryptographic issues, and its ability to integrate seamlessly into modern developer workflows. Snyk Code was installed as an extension for Visual Studio Code. Table 3.1 mentions the static analysis tools along with their versions. The results from all the tools were extracted as JSON files, which were later analyzed against the benchmark labels to obtain the results.

Tool	Version
CryptoGuard	04.05.03
CogniCrypt	3.0.2 and 4.0.1
Snyk Code	2.18.2

Table 3.1: Version of Static Analysis Tools

3.1.2 Benchmark Datasets

To assess the tools' effectiveness, we conducted thorough online searches for open-source third-party benchmarks that categorize programs according to their accurate or erroneous utilization of cryptographic APIs. We selected three datasets: CryptoAPI Bench [1], OWASP Benchmark [34], and MASC dataset [3], widely used in existing research. These benchmarks were selected for their ability to provide a comprehensive and balanced evaluation of cryptographic security. The OWASP Benchmark offers a well-balanced dataset with both true and false positive instances, allowing us to assess the accuracy of the tools on a diverse range of cryptographic issues. The MASC dataset, on the other hand, introduces mutations in the code, enabling us to evaluate whether the tools can correctly identify and address mutated instances of cryptographic misuse. Together,

these benchmarks ensure that our evaluation captures both real-world vulnerabilities and the ability to detect more nuanced or modified code patterns. These benchmarks include test cases designed to evaluate cryptographic misuses in Java.

- CryptoAPI Bench [1] consists of a total of 182 test cases, of which 181 are labeled and one unlabeled. Consequently, the unlabeled test case was excluded from the testing. Each labeled test case indicates whether it represents a true misuse case or a false misuse case. Specifically, 144 of these cases are true misuse cases, while 37 are false misuse cases.

- The OWASP Benchmark [34], serves as a Java test suite aimed at evaluating the efficacy of automated vulnerability detection methods. This benchmark consolidates vulnerabilities that have been recently identified in the Common Weakness Enumeration (CWE) [11] which is endorsed as a suitable evaluation dataset for Application Security Testing tools. We downloaded version 1.2 of this benchmark, which contains a total of 2,740 Java programs. Given that not every program employs security APIs, our analysis was limited to three specific categories: weak cryptography, weak hashing, and weak randomness. As a result, our study includes 975 programs from the original dataset, comprising 477 programs that exhibit labeled misuse of security APIs and 498 programs that illustrate proper usage.

- The MASC dataset, designed by Ami et al. [3], applies mutation testing to identify vulnerabilities in crypto detectors, assessing their ability to withstand code modifications. We manually selected 30 test cases from MASC's minimal test suites, targeting five prevalent flaw types in modern crypto detectors.

The outline of test cases for the benchmarking datasets is shown in Table 3.2. We chose to employ existing benchmark datasets rather than creating new ones for two main reasons. Firstly, these benchmarks are publicly accessible and have been carefully curated

by diverse groups of stakeholders, ensuring that our empirical comparison is both comprehensive and easily replicable by others. Secondly, specific benchmarks like OWASP Benchmark are widely recognized and hold substantial influence in the industry. The use of these datasets on static analysis tools along with LLMs concluded our finding for RQ1. Additionally, testing out LLMs with mutations helped us address RQ2.

Benchmarking Dataset	True Labels	False Labels	Total
CryptoAPI Bench	144	37	181
MASC	29	1	30
OWASP	477	498	975

Table 3.2: Labels of Benchmarking Datasets

3.1.3 Large Language Models

Among the popular LLM models, OpenAI's GPT, Google's Gemini, Meta's Llama, and Anthropic's Claude were chosen for their advanced capabilities in text-to-code conversion and sophisticated code analysis for vulnerability detection. The use of generalized LLMs mirrors the behavior of static analysis tools in that they both analyze code without execution. Just like static analysis tools that inspect code for potential vulnerabilities or misuses, these LLMs process code to identify cryptographic misuses based on patterns learned from large datasets. For each misuse instance, LLMs were provided with a standardized prompt (see Appendix A.1 for details). This prompt was designed to include details comparable to those captured by other static analysis tools, such as the method name, starting line number, highlighted message, message description (including the reason for misuse), the lines of code containing the misuse, and a label indicating whether the code involved cryptographic misuse. In all cases, the LLM response followed a similar pattern, providing the label indicating whether there was cryptographic misuse, the name of the method containing the misuse, the starting line number, the line of code,

a highlighted message for the misuse, and corresponding details explaining the cause of the misuse along with best practices to follow. The LLMs and their respective versions are listed in Table 3.3.

LLM	Version	Release Date
Meta's Llama	Llama 3.0	April 18, 2024
Open AI's GPT	gpt-4o-mini-2024-07-18	July 18, 2024
Google's Gemini	gemini-1.5-flash-002	September 24, 2024
Anthropic's Claude	Claude 3.5 Haiku	October 22, 2024

Table 3.3: Version of Large Language Models

3.2 Evaluation Metrics

To evaluate the static tools with the LLMs, the accuracy of each static tool and LLMs will be measured by these 7 metrics: ***True Positive, False Positive, False Negative, True Negative, Precision, Recall, and F-score***.

3.2.1 LLM Specific Metrics

Since Large Language Models generate text, traditional performance metrics such as precision, recall, and F-score are insufficient for fully assessing their capabilities. Therefore, in addition to these metrics, we employed two measures to evaluate security and privacy advice: ***Actionability*** and ***Specificity***.

The work of Redmiles et al. [38] inspired these metrics, who proposed an approach to evaluate the actionability and comprehensibility of privacy advice on the Web. Building upon these insights, we adapted Actionability to measure how practical and feasible the LLM-generated recommendations are for developers. This metric evaluates whether the advice is straightforward enough to be immediately applied by developers, similar to

how users benefit from clear, actionable instructions in security contexts. On the other hand, specificity was inspired by their study's comprehensibility aspect. It gauges how detailed and precise the LLM's response is, determining whether it includes clear, direct fixes, relevant references, or actionable resources for developers to effectively address a cryptographic issue. Together, these two metrics allow us to better assess the LLMs' ability to support developers in securely implementing cryptographic practices.

To evaluate Actionability, we utilized specific keywords proposed by Rostami et al. [41] to assess how well information security policies communicate actionable advice. Their research focused on identifying actionable language in security recommendations, emphasizing how certain keywords indicate the actions users should take to mitigate security risks. Based on their findings, we adopted their list of keywords to evaluate whether the response generated by the LLM was actionable. These keywords, listed in Table 3.4, include terms like "should", "need", or "must". If the LLM's response contained these keywords, it was categorized as "Actionable", as they signal clear, direct guidance for developers to address cryptographic issues.

Keywords
never
need
should
not
forbidden
must
shall

Table 3.4: Actionable Keywords

Additionally, we assessed Specificity by considering whether the LLM's response provided concrete details, such as a link to an external resource (e.g., a URL) or a reference to established frameworks like CWE (Common Weakness Enumeration) or fixed code. A response with such elements is considered more specific, providing not just general

advice but also concrete resources or actionable steps for developers to fix cryptographic misuses. By incorporating these keywords and criteria, we systematically evaluated the actionability and specificity of each LLM's response, providing a robust framework to measure how effectively LLMs can guide developers in addressing cryptographic misuses.

For example, one of the mutated test cases replaces AES with DES by replacing "A" with "D". The misuse line is as follows:

Cipher c = Cipher.getInstance("AES".replace("A", "D"));

For this particular case, GPT gave the misuse description as:
"The usage of "AES".replace("A", "D")' is a significant misuse. This expression incorrectly modifies the string that is supposed to define the algorithm name, changing it from "AES" to "DES". Since "DES" is a less secure algorithm, this can lead to vulnerabilities in the encryption implementation realized by this code. This does ***not*** *represent a proper way to specify or select cryptographic algorithms, as it can easily be manipulated or misunderstood."*

The presence of an actionable keyword "not" in GPT's response provides clear feedback to developers to fix this misuse instance.

In contrast, the absence of actionable keywords in an LLM response may limit developers' ability to understand and address the misuse effectively. For instance, GPT provided the following response in another case where AES was replaced by DES:
"The issue arises because the program allows for the selection of weak ciphers (like DES) through method chaining without any safeguards or validation mechanisms to ensure that only strong ciphers are used. This directly impacts the security posture of the application by making it susceptible to various cryptographic attacks, thereby constituting a cryptographic misuse."

Here, the lack of actionable keywords may limit the guidance that novice developers

need, reducing the response's effectiveness in directing them toward an appropriate fix and failing to emphasize the potential consequences of the misuse.

3.3 Experiments

Figure 3.1 shows our evaluation methodology for LLM-based cryptographic misuse detection. The methodology is divided into three sections: LLM vs Static Tools, Comparing LLMs, and Evaluating LLM responses.

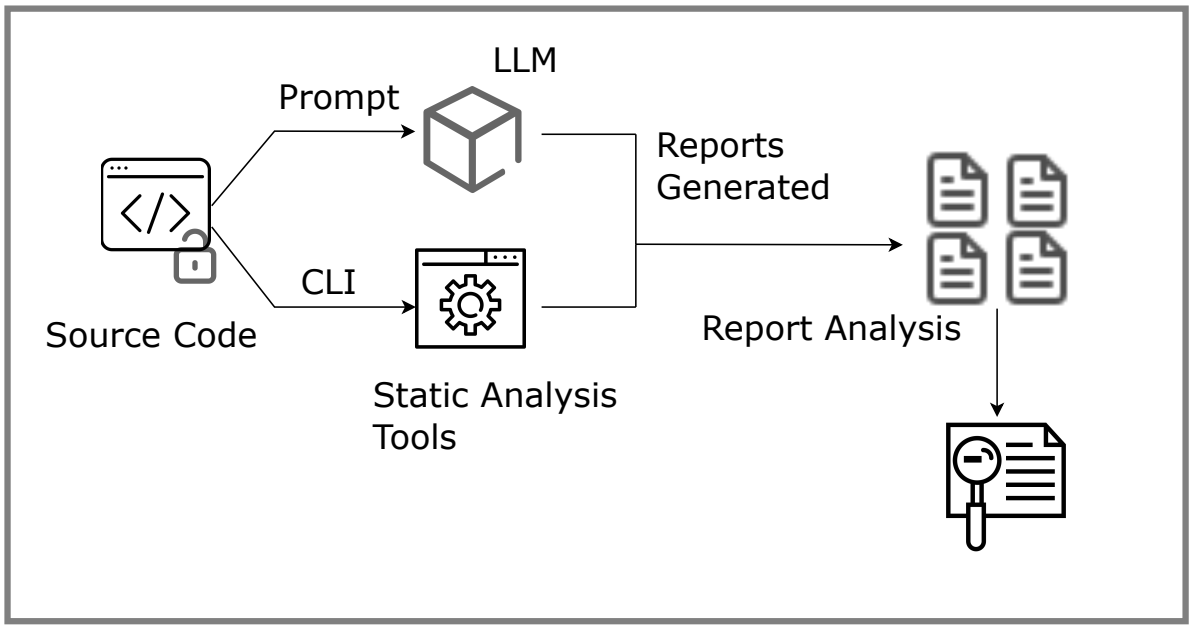


Figure 3.1: Evaluation Methodology for Comparing Static Tools with LLMs

3.3.1 LLM vs Static Tools

We began our experiments by testing LLMs and static tools with the CryptoAPI and OWASP benchmarks to evaluate their effectiveness in detecting cryptographic misuses, using the evaluation metrics outlined in the metrics section. This helped us understand how well each approach identifies cryptographic misuses and allowed us to compare the strengths of LLMs with those of static tools. This contributed to our findings on RQ1.

3.3.2 Comparing LLMs

In the second phase of our experiments, we tested LLMs using mutated test cases from the MASC dataset, along with CryptoAPI and OWASP benchmarks, to evaluate their effectiveness in detecting cryptographic misuses. This enabled us to assess the robustness of these LLMs in identifying cryptographic misuse across varied and altered scenarios, concluding our findings for RQ2.

3.3.3 Evaluating LLM Responses

Additionally, we analyzed the responses of LLMs using the Actionability and Specificity metrics. If a response from an LLM contained a keyword from Table 3.4, it was considered actionable. The number of actionable responses in a particular dataset represents the total actionability of the LLM for that dataset. To determine the Specificity of an LLM response, we manually reviewed each response to check if the LLM provided a link, referenced a CWE, or included recommended code that could assist developers in fixing the misuse. If the response from an LLM contained a link, CWE reference, or fixed code, it was considered specific. The number of test cases with specific responses in a particular dataset indicates the specificity of the LLM and is compared with other LLMs to assess specificity.

This comprehensive approach offered a clearer understanding of the responses provided by LLMs, assessing whether each response qualified as actionable advice for developers. Additionally, it revealed whether any links provided by the LLMs effectively guide developers to relevant resources for addressing specific cryptographic misuses, thus contributing insights for RQ3.

Chapter 4

Results and Findings

In this chapter, we present the results of our work on detecting cryptographic misuses for CryptoAPI and OWASP benchmarks, gathered using the selected static tools and GPT. Furthermore, we extended the results by comparing LLMs on the MASC dataset to determine which LLM performed better. We also analyzed the responses from LLMs to gain insights into which one offered more actionable and specific advice.

4.1 LLMs vs Static Tools

4.1.1 CryptoAPI Benchmark

CryptoAPI benchmark includes a total of 181 test cases, each labeled to indicate whether it represents a true misuse case or not. Specifically, 144 of these cases are true misuse cases, while 37 are false misuse cases. The performance and effectiveness of each tool in identifying cryptographic misuses are detailed below, offering insights into their strengths and limitations in practical applications.

For the CryptoAPI benchmark, GPT 4-o-mini achieved the highest F-score of 87.6%, with an impressive recall rate of 97.9%, accurately detecting 141 out of 144 misuse cases. CryptoGuard followed closely with an F-score of 84.8% and a recall of 83.3%, missing 24

misuse cases. The detailed performance metrics for each tool are presented in Table 4.1.

Benchmark	Tool	True Positive	False Positive	False Negative	True Negative	Precision (%)	Recall (%)	F Score (%)
CryptoAPI	GPT 4-o-mini	**141**	37	**3**	0	79.2	**97.9**	**87.6**
	CryptoGuard	120	**19**	24	**18**	**86.3**	83.3	84.8
	CogniCrypt 4.0.1	113	28	31	9	80.1	78.5	79.3
	CogniCrypt 3.0.2	110	31	34	6	78.0	76.4	77.2
	Snyk Code	93	30	51	7	75.6	64.6	69.7
OWASP	CryptoGuard	437	**27**	40	**471**	**94.2**	91.6	**92.9**
	GPT 4-o-mini	**477**	498	**0**	0	48.9	**100.0**	65.7
	Snyk Code	477	498	**0**	0	48.9	**100.0**	65.7
	CogniCrypt 4.0.1	259	200	218	298	56.4	54.3	55.3
	CogniCrypt 3.0.2	259	475	218	23	35.3	54.3	42.8

Table 4.1: Evaluation Metrics for CryptoAPI and OWASP Benchmark

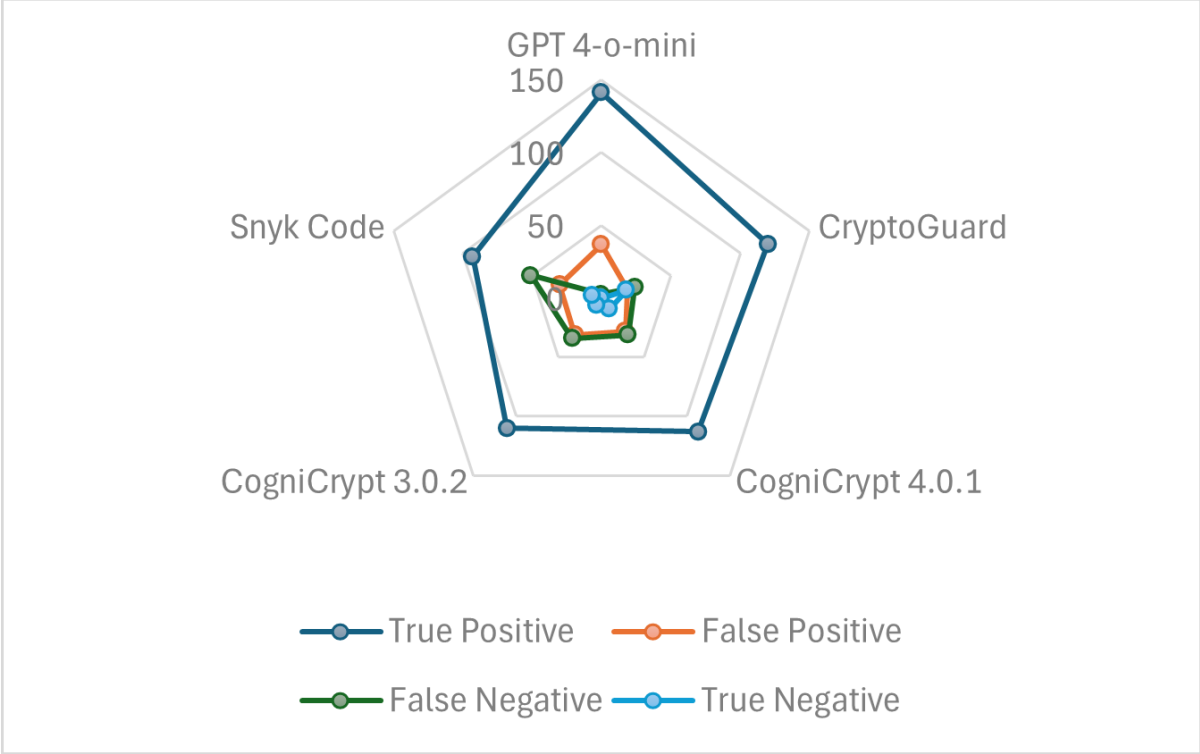


Figure 4.1: Tools Comparison on CryptoAPI Benchmark

GPT 4-o-mini's strong accuracy in detecting misuse cases highlights its potential as

an effective alternative to specialized static analysis tools for identifying cryptographic misuses. Unlike traditional tools that rely on fixed rule sets and heuristics—effective mainly for straightforward cases—GPT 4-o-mini uses advanced pattern recognition and contextual adaptability, enabling it to handle complex or ambiguous misuse cases with high precision and recall, though it has a slightly higher rate of false positives. The key metrics of the tools are highlighted in Figure 4.1 and Figure 4.2.

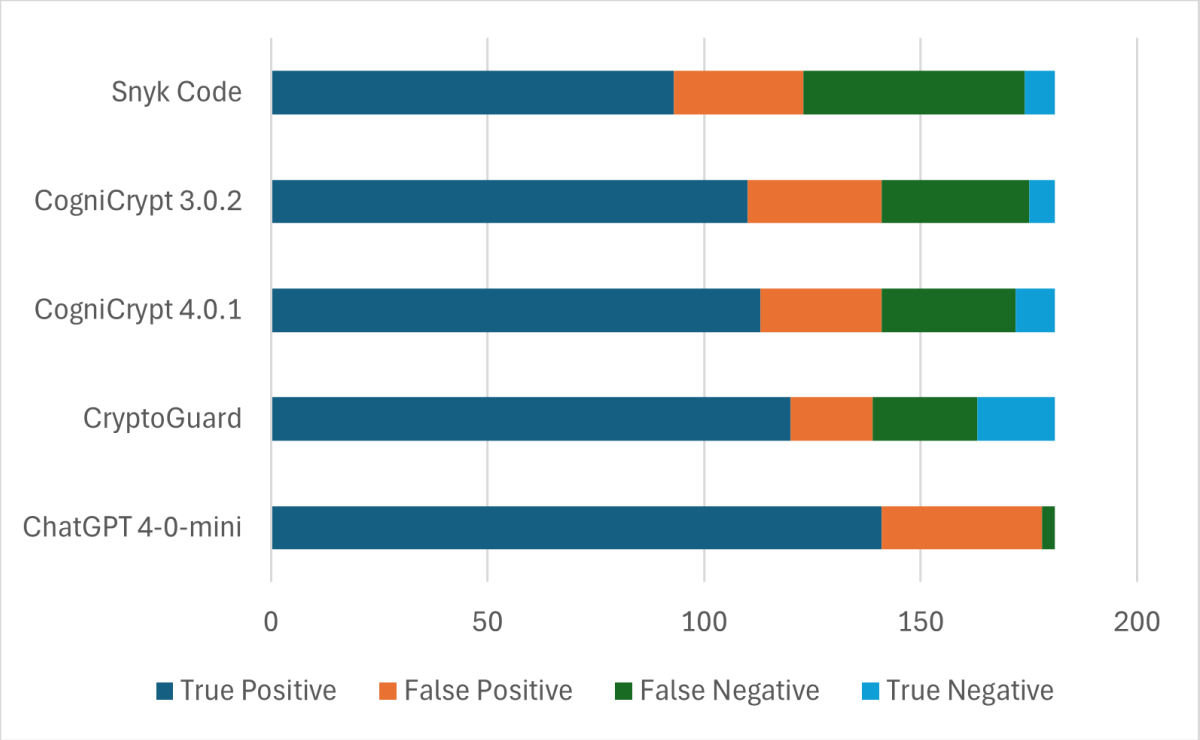


Figure 4.2: Detection Results for CryptoAPI Benchmark Across Different Tools

This flexibility allows GPT 4-o-mini to excel where static tools like CryptoGuard and CogniCrypt, limited by their predefined rule coverage and capabilities, may miss specific misuse types, such as Java's Secret Key Factory or Hostname Verifier API cases.

Finding 1 RQ1: GPT 4-o-mini achieved the highest F-score (87.6%) among tools for CryptoAPI benchmark, showing strong cryptographic misuse detection despite not being specialized, followed closely by CryptoGuard. Snyk Code and CogniCrypt had higher error rates, making them less reliable overall.

4.1.2 OWASP Benchmark

The OWASP benchmark consists of 975 test cases, each labeled to indicate whether it represents a true misuse case. Of these, 477 are true misuse cases, while 498 are false misuse cases. The performance and effectiveness of each tool in identifying cryptographic misuses are discussed, offering insights into which tool performed best against the benchmark.

CryptoGuard achieved the highest F-score of 92.9%, with a precision of 94.2% and a recall of 91.6%, indicating its ability to accurately detect true misuse cases while keeping false positives low. Snyk Code and GPT 4-o-mini both followed with an F-score of 65.7%. Although these tools had perfect recall, their high number of false positives reduced their overall accuracy, revealing strong detection capabilities but limited precision. The OWASP benchmark results are summarized in Table 4.1.

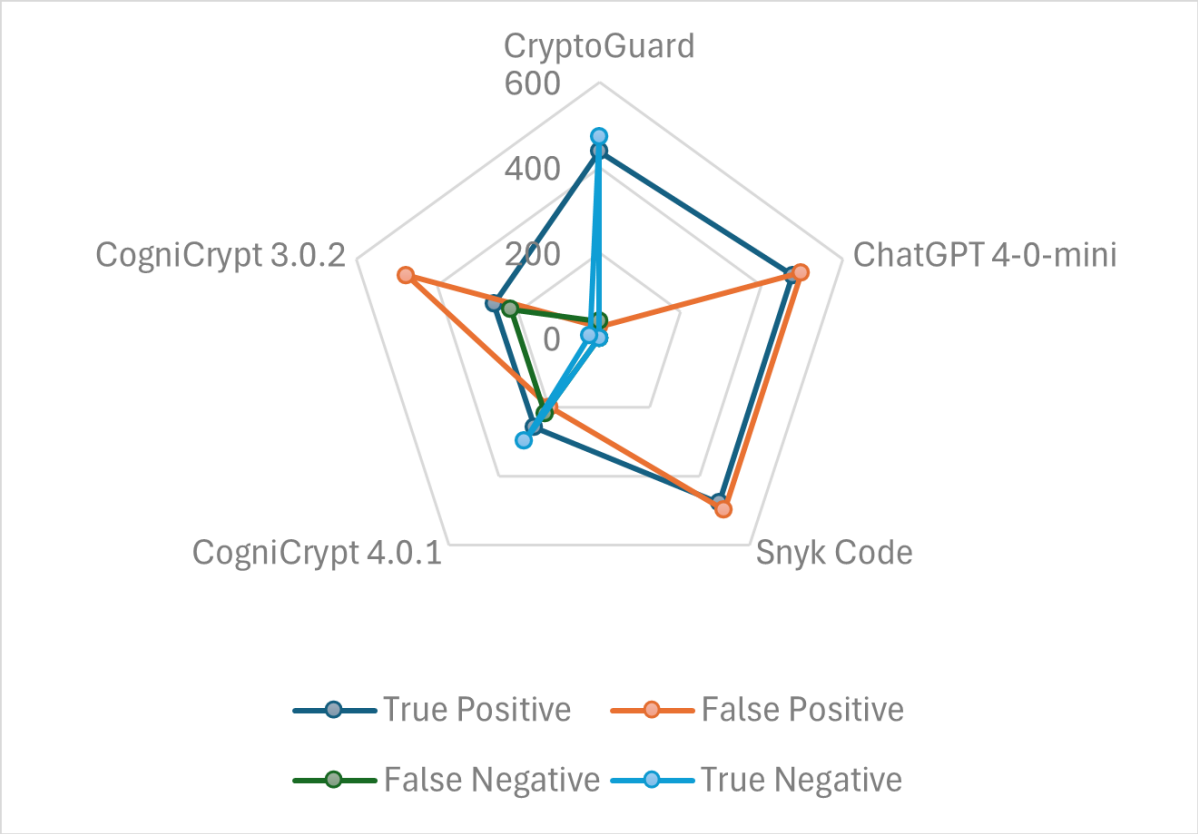


Figure 4.3: Tools Comparison on OWASP Benchmark

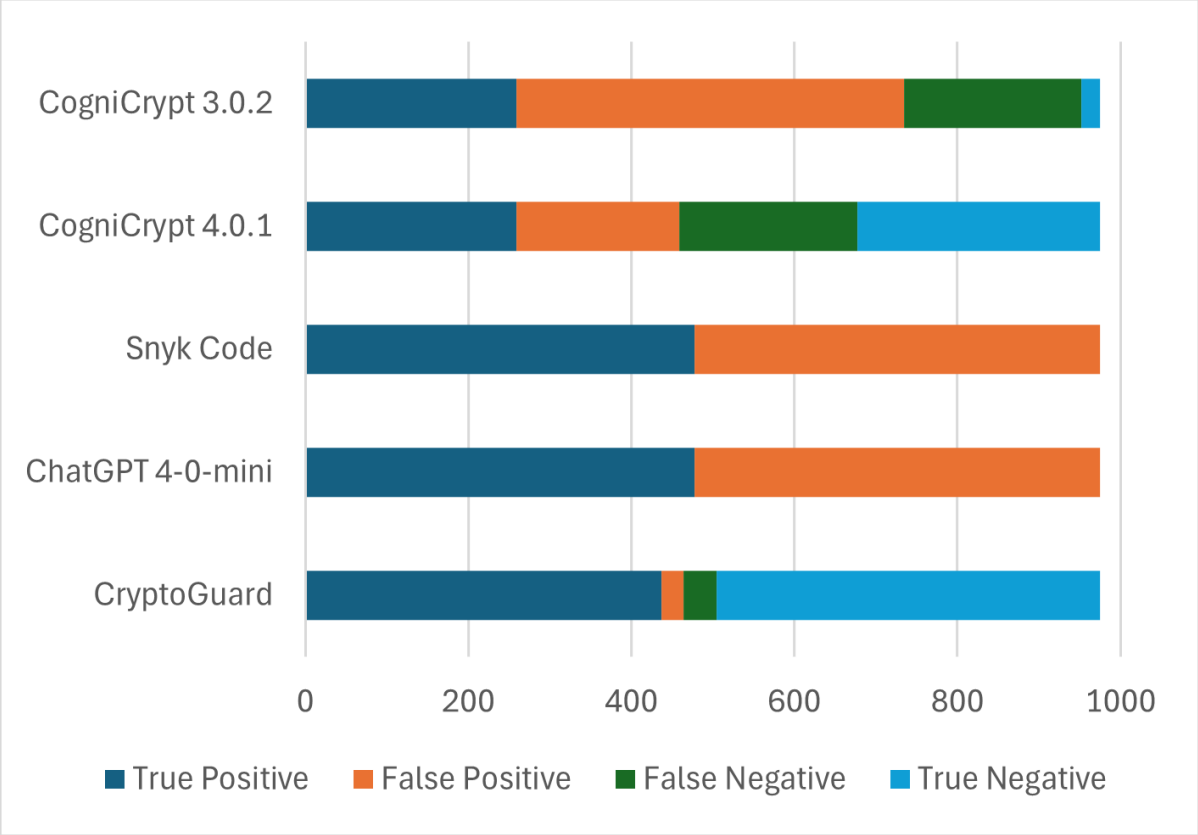


Figure 4.4: Detection Results for OWASP Benchmark Across Different Tools

GPT 4-o-mini's high false positive rate can be attributed to its text-based responses, which often suggest best practices even when no misuse is present. This tendency led GPT 4-o-mini to label cases as "TRUE" for best practice recommendations, even without actual cryptographic misuse, lowering its precision in distinguishing true misuse cases. Additionally, the OWASP test cases had a higher average line count than the CryptoAPI test cases, resulting in more code for GPT to analyze. With longer code samples, GPT tended to provide general recommendations rather than focusing solely on detecting cryptographic misuses, often labeling false cases as true. Conversely, GPT performed more accurately with smaller code samples compared to test cases with more lines of code. Figure 4.3 and Figure 4.4 display the key metrics of the tools tested on the OWASP Benchmark, highlighting CryptoGuard's superior performance over other tools.

Finding 2 RQ1: CryptoGuard outperforms other tools on the OWASP benchmark with a 92.9% F-score, while GPT and Snyk Code lag behind at 65.7%, demonstrating CryptoGuard's higher accuracy in detecting cryptographic misuses.

4.1.3 ROC Comparison

The comparison of static tools with GPT across the CryptoAPI and OWASP benchmarks showed varying performance. CryptoGuard balanced true positive detection and false positive minimization, outperforming other tools (Figure 4.5).

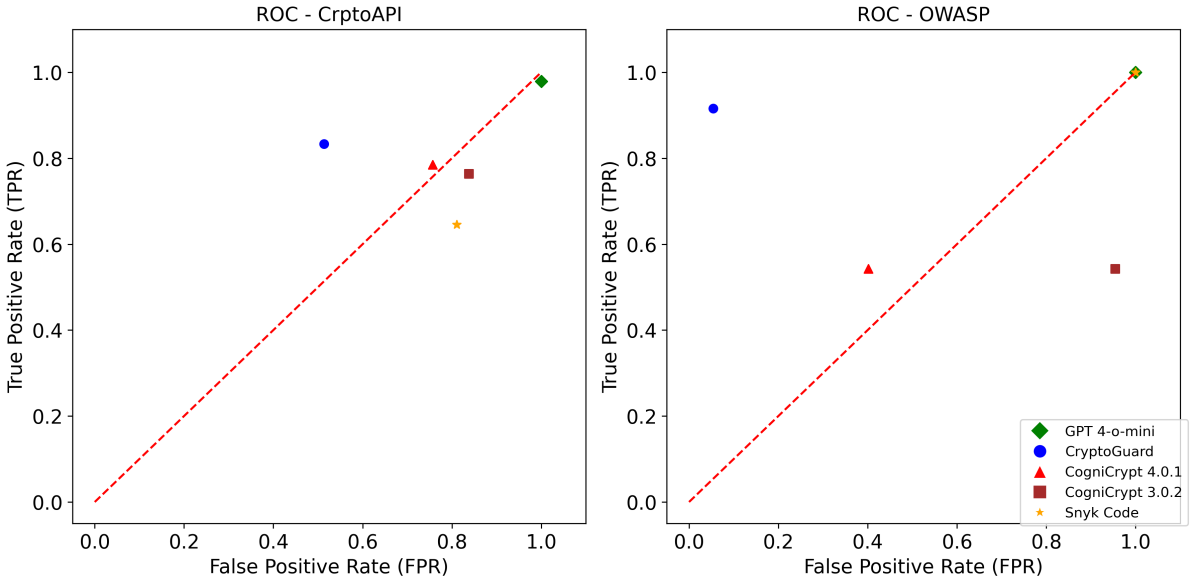


Figure 4.5: TPR vs FPR for Static Tools and GPT

GPT 4-o-mini had a high recall but a high false positive rate, highlighting the precision challenge in LLM-based vulnerability detection. In contrast, CogniCrypt and Snyk Code performed near random guessing on the CryptoAPI benchmark. CogniCrypt's limited ability to handle diverse misuse instances led to a higher false positive rate, affecting its performance. The CryptoAPI results were also influenced by the imbalance of true and false misuse instances. While LLM-based tools show promise, reducing false positives is crucial for real-world use. A limitation of this analysis is the lack of probability

mechanisms, restricting fair comparison at specific false positive rate thresholds. Future work should address this for more accurate ROC-based comparisons.

Finding 3 RQ1: CryptoGuard demonstrates a good balance between detecting true positives and minimizing false positives, outperforming other tools and GPT in some cases.

4.1.4 Time Comparison

In addition to comparing the results of each static tool with GPT, we also compared the time it took to run both benchmarks. The tools were configured on an Ubuntu virtual machine (VM) hosted on a laptop. The laptop's specifications are: 1) Operating System: Windows 11, 2) CPU: i7-1255U at 1.70 GHz, 3) RAM: 16 GB. The VMs to set up tools were created with the following specifications: 1) Operating System: Ubuntu, 2) RAM: 10 GB, 3) JVM heap size: 8 GB, and 4) Number of Processors: 4.

Tools	CryptoAPI	OWASP
CryptoGuard	1m 48s	10m 59s
CogniCrypt 3.0.2	54s	16m 35s
CogniCrypt 4.0.1	1m 7s	23m 26s
Snyk Code	**8s**	**52s**
GPT 4-o-mini	8m 45s	71m

Table 4.2: Execution Time of Tools for CryptoAPI and OWASP Benchmark

Test cases from datasets were run on GPT using an API and the time it took to get responses for each benchmark was recorded. The static analysis tools, including CryptoGuard and CogniCrypt, were run using the CLI command, whereas Snyk code was utilized as an extension to Visual Studio Code. The results of time taken for each tool for both benchmarks are highlighted in Table 4.2.

The data indicates that Snyk Code is the fastest tool across both the CryptoAPI and OWASP benchmarks, with execution times of 8 seconds and 52 seconds, respectively. CryptoGuard also performs efficiently, especially on the CryptoAPI benchmark (1m 48s), though it takes longer on the OWASP benchmark (10m 59s). In contrast, GPT 4-o-mini has the longest execution times, requiring 8m 45s for the CryptoAPI and 71 minutes for the OWASP benchmark, highlighting a notable delay compared to other tools. This suggests that while GPT 4-o-mini may excel in detection accuracy, its slower processing speed could limit its scalability for large datasets.

Finding 4 RQ1: Snyk Code exhibits the fastest execution time across both benchmarks, followed by CryptoGuard with the second-best performance. In contrast, GPT 4-o-mini, despite its detection strengths, has significantly slower processing speeds, which may hinder its scalability for larger code bases.

4.2 Comparing LLMs with Datasets

For this section, we evaluated the performance of the selected LLMs against the benchmark datasets. We also presented the results of running mutated test cases to determine which model performed better. GPT 4-o-mini was accessed via an API, while Gemini 1.5 Flash, Llama 3.0, and Claude Haiku 3.5 were accessed through its chat interface. For the CryptoAPI and OWASP benchmarks, Gemini, Llama, and Claude were tested on 30 random test cases, and the scores were compared with GPT to determine which performed better. Similarly, LLMs were tested on 30 manually curated MASC test cases to conclude which LLM performed better. Examples of some LLM responses are provided in Appendix A.2.

To compare GPT with other LLMs, 30 test cases from the CryptoAPI and OWASP benchmarks were randomly selected and evaluated using the same prompts to ensure consistency. On the CryptoAPI benchmark, GPT achieved the highest F-score of 87.6% with a recall of 97.9%, followed closely by Claude, which achieved perfect recall (100%) but a slightly lower F-score of 86.8%. Overall, the F-scores of all LLMs were comparable, with Gemini recording the lowest F-score of 83.3%. For the OWASP benchmark, GPT again led in overall performance with an F-score of 65.7%, closely followed by Gemini with an F-score of 63.2%. Llama and Claude tied for the lowest performance, each achieving an F-score of 57.1%. Notably, all the LLMs demonstrated perfect recall (100%). These results underscore GPT's dominance in both recall and F-score across the benchmarks, with the other models performing comparably but trailing slightly behind (see Table 4.3).

On the mutation dataset (MASC), which tests robustness, GPT and Claude were the best performers, both achieving a recall of 100% and an F-score of 98.3% by correctly identifying all 29 true misuse cases. In contrast, Gemini was the weakest, with a recall of 79.3% and an F-score of 88.5%, detecting only 23 of 29 misuse cases. This demonstrates the strength of LLMs in varied scenarios, an area where many existing static tools still

Benchmark	LLM	True Positive	False Positive	False Negative	True Negative	Precision (%)	Recall (%)	F Score (%)
CryptoAPI	GPT 4-o-mini	**141**	37	3	0	79.2	98	**87.6**
	Llama 3.0	22	7	1	0	75.9	95.7	84.6
	Claude 3.5 Haiku	23	7	**0**	0	76.7	**100**	86.8
	Gemini Flash 1.5	20	**5**	3	**2**	**80.0**	87	83.3
MASC	GPT 4-o-mini	**29**	1	**0**	0	96.7	**100**	**98.3**
	Llama 3.0	28	1	1	0	96.6	96.6	96.6
	Claude 3.5 Haiku	29	1	**0**	0	96.7	**100**	**98.3**
	Gemini Flash 1.5	23	**0**	6	**1**	**100**	79.3	88.5
OWASP	GPT 4-o-mini	**477**	498	**0**	0	**48.9**	**100**	**65.7**
	Llama 3.0	**12**	18	**0**	0	40	**100**	57.1
	Claude 3.5 Haiku	**12**	18	**0**	0	40	**100**	57.1
	Gemini Flash 1.5	**12**	**14**	**0**	**4**	46.2	**100**	63.2

Table 4.3: Comparison of LLMs on Different Benchmarks[1]

face challenges.

Finding 1 RQ2: GPT demonstrates superior performance in detecting cryptographic misuses across all benchmarks, achieving higher recall and F-score than other LLMs. However, this accuracy comes at the cost of a higher rate of false positives. As identifying true misuse is more critical than minimizing false positives, GPT is the more effective LLM for detecting cryptographic misuse.

4.2.1 ROC Comparison

Among LLMs, Gemini 1.5 Flash performs better than the others, correctly categorizing false instances as false and falling to the left of the random guessing line (Figure 4.6 to Figure 4.8).

In contrast, GPT-4-o-mini, Llama 3.0, and Claude 3.5 Haiku are closer to the random guessing line, correctly identifying true instances but misclassifying false ones. These

[1]Gemini, Llama, and Claude were evaluated against random 30 test cases for CryptoAPI and OWASP datasets.

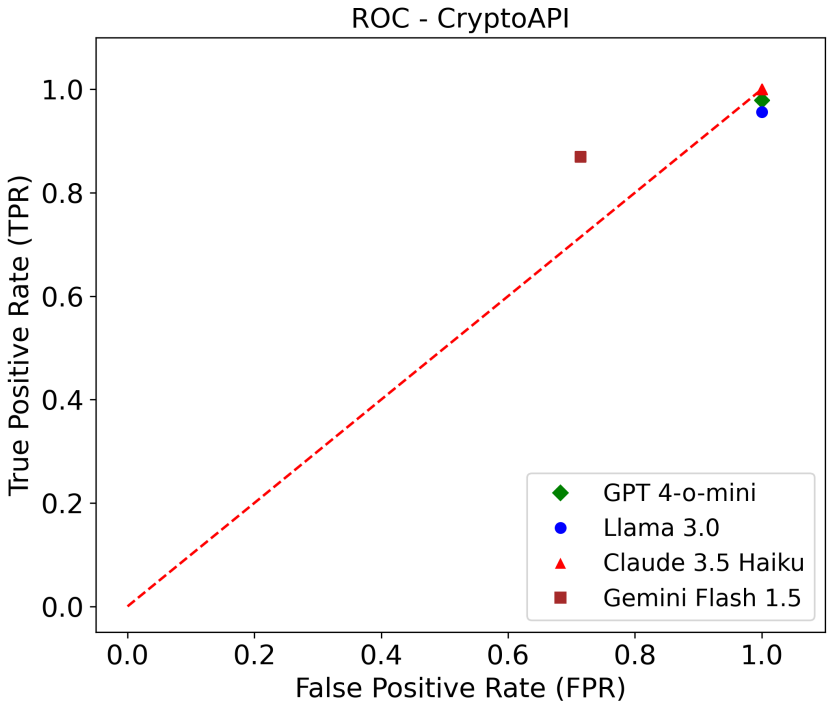


Figure 4.6: TPR vs FPR of LLMs for CryptoAPI

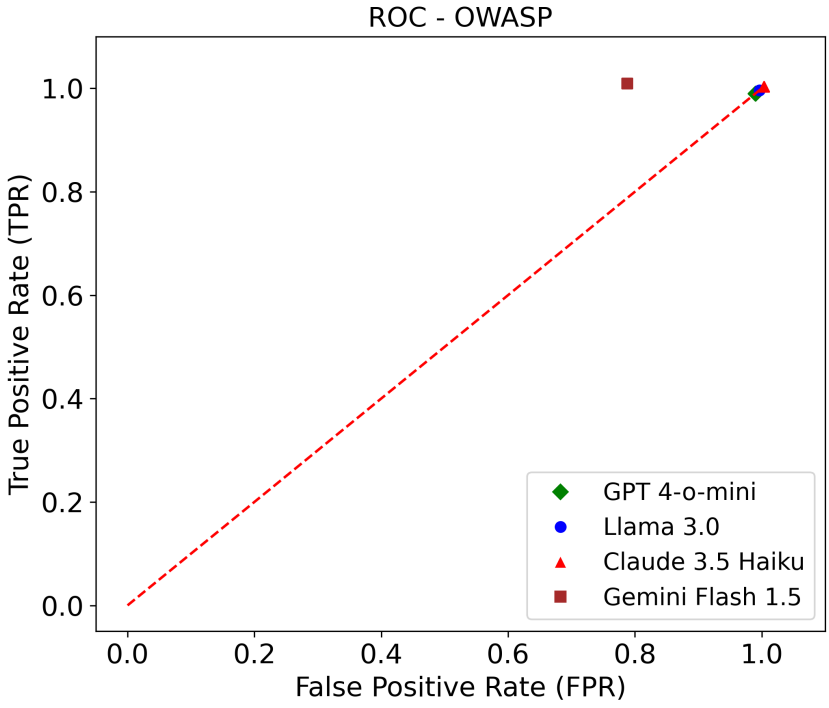


Figure 4.7: TPR vs FPR of LLMs for OWASP

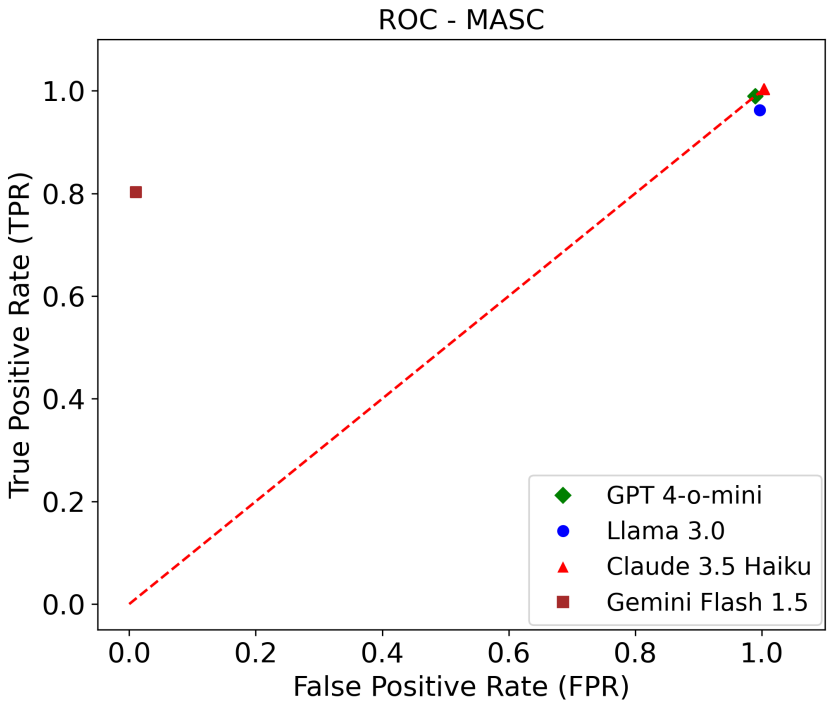


Figure 4.8: TPR vs FPR of LLMs for MASC

results are influenced by a class imbalance in MASC, where true instances dominate, and only a single false instance is present. Additionally, the results are affected by the fact that, apart from GPT, all other LLMs were tested against only a subset of misuse instances from the CryptoAPI and OWASP benchmarks. While LLMs show potential, reducing false positives and addressing class imbalance remain crucial for ensuring a fair comparison.

Finding 2 RQ2: Gemini strikes a good balance between detecting true positives and minimizing false positives, although GPT and other LLMs outperform it with higher recall and an improved F1 score.

4.3 Evaluating LLM Responses

To evaluate the responses of LLMs, we used two metrics: Actionability and Specificity. Actionability was assessed by examining each LLM response to check for the presence of an actionable keyword. If an actionable keyword was included, the response was marked as actionable; otherwise, it was marked as non-actionable. The percentage of actionable cases for each benchmark was then calculated and compared across LLMs. Similarly, a response was considered specific if it included a reference to the CWE, a link to an external resource, or provided fixed code for a misuse instance; otherwise, it was marked as non-specific. The percentage of specific cases provided a measure of specificity for each benchmark. The results for actionability and specificity across benchmarks are presented in Table 4.4.

Benchmark	LLM	Actionable Responses	Specific Responses	Total Responses	Actionability (%)	Specificity (%)
CryptoAPI	GPT 4-o-mini	162	0	264	**61.4**	0.0
	Llama 3.0	14	2	30	46.7	**6.7**
	Claude Haiku 3.5	16	2	30	53.3	**6.7**
	Gemini Flash 1.5	9	2	30	30.0	**6.7**
MASC	GPT 4-o-mini	19	0	30	**63.3**	0.0
	Llama 3.0	15	2	30	50.0	6.7
	Claude Haiku 3.5	15	14	30	50.0	**46.7**
	Gemini Flash 1.5	7	7	30	23.3	23.3
OWASP	GPT 4-o-mini	1584	0	1909	83.0	0.0
	Llama 3.0	27	0	30	**90.0**	0.0
	Claude Haiku 3.5	26	13	30	86.7	**43.3**
	Gemini Flash 1.5	21	6	30	70.0	20.0

Table 4.4: Actionability and Specificity of LLMs Across Benchmark Datasets

GPT 4-o-mini generated the most actionable responses, with 63.3% actionability on MASC and 61.4% on CryptoAPI, but lacked specificity, scoring 0% across all benchmarks.

Llama 3.0 demonstrated the highest actionability for OWASP, achieving 90.0%, though it also lacked specificity. Claude 3.5 Haiku excelled in specificity, with 43.3% on OWASP and 46.7% on MASC, while maintaining strong actionability of 86.7% on OWASP and 53.3% on CryptoAPI. These results suggest that GPT 4-o-mini excels in actionability for CryptoAPI and MASC, while Llama 3.0 leads in actionability for OWASP with 90.0%. Claude 3.5 Haiku, although slightly less actionable, offers the best specificity by referencing CWEs, external resources, or fixable code to address cryptographic misuses.

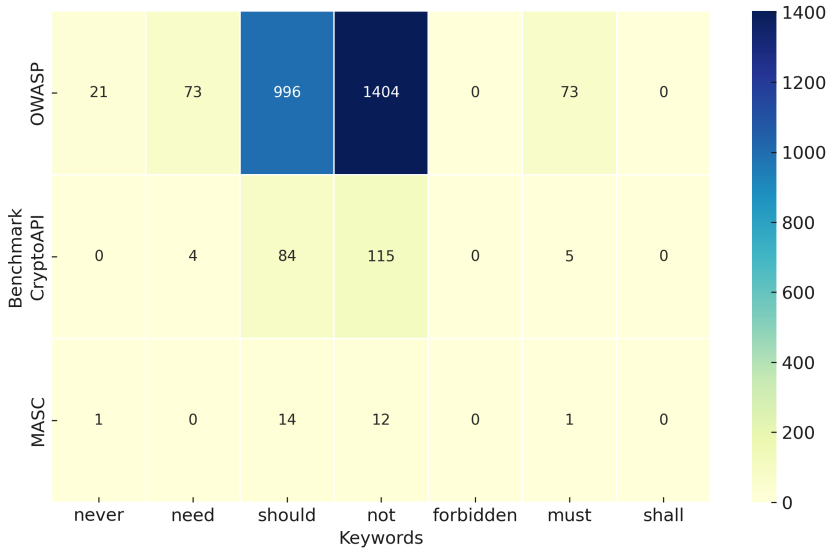


Figure 4.9: GPT Actionable Keywords Occurrence

Figures 4.9 to 4.12 show the occurrence of actionable keywords for the selected LLMs. The most frequently used keywords across benchmarks are “not” and “should”. In both the OWASP and CryptoAPI benchmarks, “should” is common, with 996 instances in GPT for OWASP, and a lower but consistent presence across other benchmarks, providing positive actionable feedback for developers. Similarly, “not” appears frequently, especially in OWASP, with 1404 instances in GPT, providing negative actionability and guiding developers on cryptographic misuses. “Forbidden” and “shall” were the least used keywords across different benchmarks for the selected LLMs, highlighting a focus on suggestion and negation, guiding actions to consider or avoid. Less frequent keywords, such

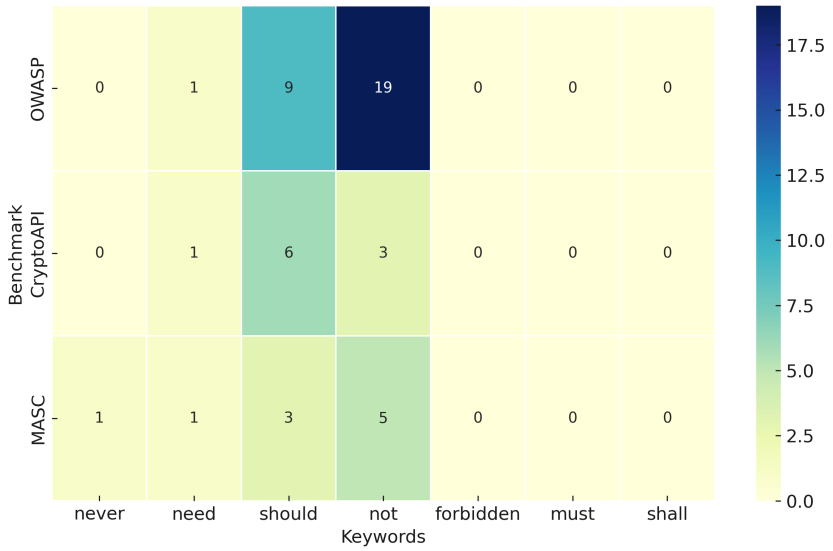


Figure 4.10: Gemini Actionable Keywords Occurrence

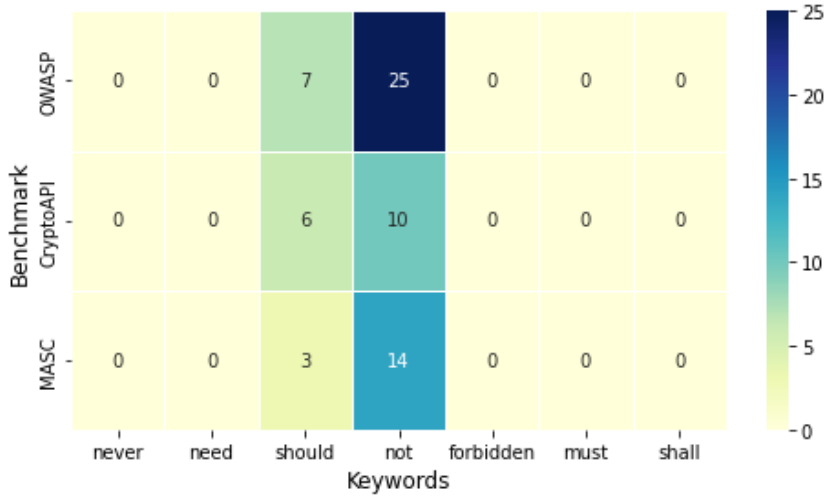


Figure 4.11: Llama Actionable Keywords Occurrence

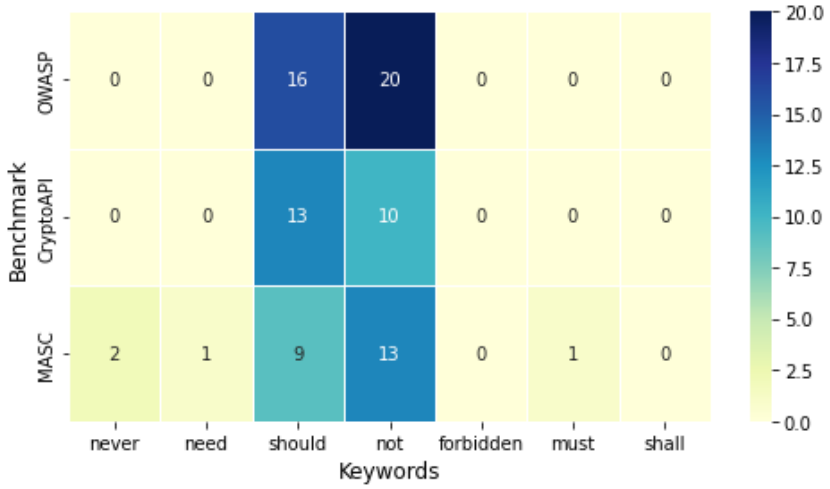


Figure 4.12: Claude Actionable Keywords Occurrence

as "need", "must", "never" and "shall", suggest a preference for softer language rather than strict requirements.

Finding RQ3: GPT 4-o-mini excels in actionability across multiple benchmarks, providing actionable feedback on things to consider and avoid, but lacks specificity. In contrast, while Claude 3.5 Haiku exhibits lower actionability, it provides greater specificity compared to other LLMs.

Chapter 5

Discussion and Ecological Validity

5.1 Discussion

5.1.1 F-score vs Recall

The F-score used to analyze the most effective tool may not be the best measure for evaluating its effectiveness in detecting cryptographic misuses. A key issue with static tools is their tendency to produce a high number of false positives, which significantly impacts the F-score and helps determine which tool performed better. However, for security purposes, recall is the most important measure alongside the F-score, ensuring that the tool does not miss any true misuse instances. Although Snyk Code and GPT have significantly lower F-scores on the OWASP benchmark compared to Cryptoguard, it achieves a perfect recall of 100%, indicating that it did not miss any true instances. GPT and Snyk Code's conservative approach resulted in a high number of false positives, which impacted its F-score.

While the F1-score is an effective measure for evaluating a tool's performance, it should not be the only criterion for tool selection. In a security context, placing more emphasis on recall will provide more desirable results than relying solely on the F-score. For the OWASP benchmark, calculating the F2-score significantly improves the score of

Tool	F1 Score	F2 Score
CryptoGuard	92.9	92.1
GPT 4-o-mini	65.7	82.7
Snyk Code	65.7	82.7
CogniCrypt 4.0.1	55.3	54.7
CogniCrypt 3.0.2	42.8	49.0

Table 5.1: Comparison of F-1 and F-2 Scores for OWASP

Snyk code and GPT from approximately 66% to 83%, highlighting the tool's effectiveness in detecting true alerts.Table 5.1 presents the results for the F2-score, which assigns more weight to recall than precision.

5.1.2 Strengths of Large Language Models

LLMs have an inherent advantage over static analysis tools as static tools rely on a rule-based approach. Any deviations of misuse instance from a rule will likely cause the tool to miss it. However, LLM detection capability is not particular to a specific rule set, and it can detect misuse instances in a variety of code as highlighted by the results of mutated test cases. Similarly, LLMs are not specific to a certain programming language. It can read and understand various programming languages as opposed to static analysis tools that are specific to a programming language. LLMs do not have a framework dependency and now integrations are being supported to adopt LLMs for various frameworks which will likely increase its usage in software development. Even though LLMs currently have a high false positive rate for detecting cryptographic misuses, this is likely to improve as they are trained on more data. Better accuracy than static analysis tools in detecting true alerts for security context shows LLMs can be utilized in this domain.

5.1.3 Actionability vs. Specificity in LLM Responses

We introduced LLM-specific metrics—Actionability and Specificity—to better evaluate how effectively LLMs assist in detecting and resolving cryptographic misuse instances.

Actionability refers to how well an LLM response provides clear, practical steps to address a misuse, while Specificity measures whether the response includes precise details, such as a corrected code snippet, references to Common Weakness Enumeration (CWE), or links to external resources that guide developers toward a solution.

The balance between actionability and specificity plays a crucial role in determining which LLM is more beneficial depending on the developer's expertise level and the specific use case. For instance, mid-level or experienced developers with prior knowledge of cryptographic principles may find higher specificity more useful. These developers already understand general security practices, so responses containing fixed code, CWE references, or external resources allow them to validate and refine their implementations more efficiently. A highly specific response reduces ambiguity and helps ensure adherence to industry standards.

Conversely, novice developers or those unfamiliar with cryptographic best practices may benefit more from an LLM that prioritizes actionability. A more actionable response not only flags a misuse but also explains why it is problematic, outlines the risks involved, and provides clear, structured guidance on how to correct it. This approach is particularly valuable in educational contexts or in situations where security knowledge gaps exist, as it helps developers build foundational understanding and reinforces best practices.

Therefore, the choice between an LLM that is more actionable versus one that is more specific depends on the intended audience and use case. LLMs with higher actionability are more suitable in learning environments or for developers who require step-by-step guidance, whereas LLMs with higher specificity are better suited for experienced professionals who need precise, detailed information to refine secure implementations. Striking the right balance between these two metrics can enhance the effectiveness of LLM-assisted cryptographic misuse detection and remediation.

5.1.4 Benchmark Inconsistencies

Our analysis revealed inconsistencies in the CryptoAPI benchmark, where the label sheet, which indicates whether instances are true or false, includes entries for test cases not present in the dataset. Specifically, we found labels for CredentialInStringBBCase2.java and PredictableSeedsABPMCase2.java in the label sheet, although these Java files are missing from the dataset. This mismatch can increase false negatives, potentially giving a misleading impression of the tools' effectiveness. Additionally, the dataset includes Java test cases without corresponding labels, causing tools to detect misuse instances without a reference. To maintain consistency in this study, we disregarded these unlisted test cases. This highlights the need for a comprehensive, standardized dataset to validate tool effectiveness across all Java API categories.

5.1.5 LLM Hallucination

One of the key challenges in applying LLMs to security tasks is their tendency to hallucinate, generating incorrect or misleading information with high confidence. In the context of cryptographic misuse detection, hallucinations can manifest in several ways, such as flagging secure implementations as vulnerable, suggesting non-existent vulnerabilities, or recommending incorrect remediations. This not only contributes to a high false positive rate but also erodes trust in AI-assisted security tools, as developers and security professionals may struggle to distinguish valid findings from fabricated ones. Unlike traditional static analysis tools, which operate on well-defined rules and heuristics, LLMs generate responses based on probabilistic patterns, making it difficult to ensure correctness and consistency. Addressing hallucination in security-related LLM applications requires improved fine-tuning, confidence calibration, and verification mechanisms to ensure that the information provided by these models is both accurate and actionable. Without mitigating hallucinations, LLMs may struggle to gain acceptance as reliable security tools,

particularly in environments where misinterpretations can lead to significant security failures. Although this challenge was not encountered in our work, LLMs consistently provided expected responses, though with a high false positive rate.

5.2 Ecological Validity

5.2.1 Applicability to Real World Scenarios

Our experiments used the OWASP Benchmark, a widely recognized open-source Java suite designed to evaluate application security tools. This benchmark is commonly used in both research and industry to test how well tools detect vulnerabilities in a controlled yet realistic setting. By including it in our study, we ensured that our evaluation aligns with established security research practices and that our results can be compared with existing tools and studies. Since many static analysis tools are tested against the OWASP Benchmark, using it helps validate our findings in a meaningful way.

We also included the CryptoAPI benchmark, which has been widely used in previous research for assessing cryptographic security. Cryptographic vulnerabilities often arise from improper API usage rather than flaws in the cryptographic algorithms themselves. The CryptoAPI benchmark provides structured test cases that reflect real-world mistakes developers make when implementing cryptographic functions. By testing LLMs against this dataset, we were able to evaluate their ability to detect these misuses and compare their performance with existing tools that have been benchmarked in similar studies.

To capture a broader range of coding practices, we also used the MASC dataset, which includes code written in different styles and structures. Unlike benchmarks that follow strict patterns, MASC introduces variations in how developers write code, which is important when testing LLMs. Since LLMs process code differently depending on formatting, logic, and structure, using MASC helps assess whether these models can handle diverse real-world scenarios. This makes the study more practical by considering the nat-

ural variation found in developer-written code rather than relying solely on structured benchmarks.

The results from these benchmarks provided insights into how well different tools and LLMs detect vulnerabilities in various contexts. Since our focus was on cryptographic misuses, using established benchmarks allowed us to compare our findings with existing security tools, both from industry and academia. Because these datasets are commonly used, they serve as a reliable measure of accuracy in detecting cryptographic flaws. The fact that our results are based on widely accepted benchmarks makes them more applicable to real-world security assessments rather than just controlled test cases.

Beyond benchmark testing, the way LLMs are being adopted by developers, especially novices, makes this work even more relevant. Many developers now rely on AI-powered coding assistants, like ChatGPT and GitHub Copilot, directly within their coding environments, often replacing traditional searches and documentation. Instead of looking up security best practices, developers increasingly trust these models to generate correct and secure code. This means that the accuracy of LLM-generated security recommendations has a direct impact on real-world applications. If these models frequently produce false positives or suggest insecure implementations, it could lead to security issues at scale.

Because of this shift, it's important to understand whether LLMs provide reliable security guidance, especially when dealing with cryptography. If developers unknowingly implement insecure cryptographic practices suggested by LLMs, it could introduce vulnerabilities that go unnoticed until they become serious security risks. Our study highlights these concerns by evaluating whether LLMs can accurately detect cryptographic misuses and provide actionable guidance. As AI coding assistants become more common, improving their security reliability will be essential to ensuring that they help developers write secure, not just functional, code.

The application of LLMs in real-world scenarios can be concretely observed through the integration of AI-driven tools with existing code-linting and static analysis systems.

For example, code-linting tools like ESLint or CheckStyle, often used by developers to enforce coding standards, could be augmented with cryptographic misuse detection capabilities powered by LLMs. These tools could automatically flag insecure cryptographic implementations, such as the use of ECB mode for encryption or weak key management practices. With LLMs integrated into these environments, developers could receive more contextually aware suggestions, like recommending AES-GCM over ECB mode, with explanations of why it's a more secure choice. This immediate feedback during coding would help developers avoid cryptographic vulnerabilities at the earliest stage of development.

Additionally, tools like SonarQube or Snyk, which are commonly integrated into the CI/CD pipeline for continuous integration and deployment, could greatly benefit from LLM-based detection approaches. These tools are designed to scan the code for vulnerabilities and typically rely on static analysis techniques. Although these tools are effective at identifying well-known vulnerabilities, they may not always be able to understand the deeper context of cryptographic code. For example, static tools might flag the use of outdated or insecure algorithms like DES without providing detailed recommendations on secure alternatives. In contrast, an LLM-based approach could analyze the code in real-time, understanding the context of the cryptographic functions, and providing more precise, actionable feedback, such as recommending AES-GCM and explaining why it is a more secure option.

Integrating LLM-based detection into the CI/CD pipeline means that cryptographic issues are automatically flagged before deployment, helping teams enforce secure coding practices continuously. This reduces the risk of introducing security flaws into the production code and ensures that cryptographic vulnerabilities are addressed as part of the development lifecycle, with minimal manual effort required from developers.

In this way, the combination of AI-powered tools with static analysis systems such as ESLint, CheckStyle, SonarQube, and Snyk could provide a more context-aware, dynamic, and actionable approach to cryptographic misuse detection, ensuring that developers are

not only identifying security flaws but also receiving the right guidance to fix them. This continuous feedback loop can enhance developers' security knowledge and help them implement more secure coding practices, ultimately improving the overall security of applications in the real world.

5.2.2 Threats to Validity

Threats to Internal Validity The scope of our findings may be limited by the specific tools and datasets chosen for testing. To address this, we plan to expand our evaluation by including additional tools and testing them on examples drawn from real-world cryptographic code. Another potential limitation is data leakage, as the open-source benchmarks used for testing could have been used in the training of LLMs, potentially affecting our results; however, the high number of false positives suggests otherwise. In the future, we plan to evaluate test cases by anonymizing class names to prevent any possible data leakage. LLMs may hallucinate non-existent vulnerabilities or incorrect patterns, leading to false positives that require manual verification. Moreover, the actionability and specificity metrics evaluated in our study may not fully translate into practical guidance for developers to fix code in every instance.

Threats to External Validity Our results may not generalize beyond the specific tools and datasets used in this study. To address this, we plan to expand our evaluation by incorporating additional tools, including those used in industry, and real-world cryptographic benchmarks, to test the robustness of our findings. Additionally, our findings are specific to Java and may not apply in the same way to other programming languages. Since the cryptographic misuse detection methods were tailored to Java-specific constructs and patterns, the results might not hold true for other languages, where language-specific features and cryptographic implementations can vary. Furthermore, we focused on detecting mutations in test cases that were specific to Java as a programming language. The accuracy of detecting mutant misuses could therefore differ in other

programming languages, where different constructs and syntaxes might require tailored mutation strategies.

Chapter 6

Future Work and Conclusion

6.1 Future Work

Future research could explore testing LLMs on real-world codebase benchmarks, such as the Apache CryptoAPI benchmark, to assess performance on complex cryptographic scenarios. While synthetic benchmarks and small-scale test cases provide controlled environments for evaluation, real-world codebases introduce challenges such as inconsistent coding styles, undocumented security practices, and dependencies across multiple files.

Conducting evaluations on large-scale repositories like Apache or GitHub projects would allow researchers to measure how well LLMs handle practical cryptographic misuse cases embedded within real software systems. This could involve analyzing whether LLMs can correctly identify, explain, and suggest remediations for cryptographic vulnerabilities in realistic settings, bridging the gap between academic evaluation and industry needs.

Expanding the study to include newer versions of LLMs, particularly those specialized in code generation and code fixing, may provide insights into the benefits of using code-focused models over general-purpose ones. Recent advancements in AI-driven coding assistants, such as those fine-tuned for static analysis or vulnerability detection, could significantly improve detection rates and reduce false positives. Future research could

compare how general-purpose LLMs perform against specialized models trained specifically for software security. A progressive study could also be conducted to track improvements in LLM-based detection over successive versions, identifying whether newer iterations, such as GPT-4o, demonstrate better contextual understanding of cryptographic misuse patterns.

Another potential avenue for future research is the generation of more refined ROC curves by evaluating model predictions across different thresholds. A possible strategy could involve leveraging a pass-at-k mechanism, where each test case is run multiple times (e.g., three or five iterations) to assess whether the model consistently predicts the correct outcome. By aggregating these results and adjusting decision thresholds dynamically, researchers can create more realistic ROC curves that better reflect the model's performance across varying levels of confidence. This approach could help in determining whether iterative testing reduces the false positive ratio, leading to a more accurate assessment of an LLM's reliability in cryptographic misuse detection.

Additionally, testing LLMs on benchmarks in other programming languages could reveal language-specific strengths and limitations. Since different programming languages enforce varying cryptographic libraries, APIs, and security paradigms, an LLM's effectiveness might differ depending on the syntax, semantics, and available security best practices of a given language. For instance, Java's strict exception handling and extensive cryptographic API documentation may present different challenges compared to the flexibility of Python's dynamic typing and widespread use of third-party libraries.

Moreover, conducting cross-language comparisons could uncover whether certain LLMs exhibit bias toward more commonly trained languages and whether they struggle with less popular or lower-resourced programming ecosystems. This investigation would be particularly beneficial for multilingual development teams and organizations maintaining software in multiple languages, as it would help in selecting the most suitable AI-driven security tool for their needs.

6.2 Conclusion

Cryptographic misuses in software systems pose serious security risks, and addressing these vulnerabilities during the development process is crucial to protecting sensitive data. Our work compared the capabilities of Large Language Models (LLMs) with traditional static analysis tools in detecting cryptographic misuses. Static analysis tools have long been employed to detect such vulnerabilities, but with the rise of AI-powered systems, particularly LLMs, there is an increasing interest in evaluating their potential to automate security analysis tasks. Our study sought to evaluate how LLMs perform when tasked with detecting cryptographic misuses across well-established cryptographic benchmarks, including CryptoAPI, OWASP, and MASC.

Through our experiments, we found that GPT demonstrated strong performance, often surpassing traditional static analysis tools in terms of accuracy when applied to the CryptoAPI and MASC datasets, however, CryptoGuard outperformed GPT and other static analysis tools for OWASP. This indicates that LLMs have the potential to identify a broader set of cryptographic misuses compared to some static tools. However, a significant issue that emerged from our evaluation was the relatively high rate of false positives. Despite the strong detection capabilities, LLMs frequently flag code as potentially insecure, which presents a serious challenge for adoption in real-world environments.

GPT demonstrated strong performance, often surpassing traditional static analysis tools in terms of accuracy when applied to the CryptoAPI and MASC datasets. However, CryptoGuard outperformed GPT and other LLMs for the OWASP dataset, highlighting the varying strengths of each tool depending on the dataset and the type of misuse being detected. This indicates that LLMs have the potential to identify a broader set of cryptographic misuses compared to some static tools. However, a significant issue that emerged from our evaluation was the relatively high rate of false positives. Through our experiments, we found that no tool or LLM was universally better than others across all aspects of cryptographic misuse detection.

Despite the strong detection capabilities, LLMs frequently flag code as potentially insecure, which presents a serious challenge for adoption in real-world environments. The issue of false positives is particularly concerning in the context of cryptographic security. Security teams and developers may be overwhelmed by the number of alerts generated, leading to fatigue and increased time spent manually verifying and filtering out irrelevant findings. This may, in turn, undermine the effectiveness of LLM-driven tools, as the core advantage of automating security tasks—improving efficiency and reducing the burden on security professionals—would be diminished. To mitigate this challenge, reducing false positive rates is essential for ensuring that LLMs become a practical tool for security analysis.

In our assessment, we went beyond conventional metrics such as precision, recall, and F-score by introducing two evaluation metrics: Actionability and Specificity. These metrics allowed us to measure not just how well LLMs identified cryptographic misuses, but how actionable and specific their advice was in real-world scenarios. Our findings showed that while GPT was more actionable than other LLMs for the CryptoAPI and OWASP datasets, Llama showed higher actionability for OWASP specifically. Additionally, while GPT provided clear recommendations, Claude demonstrated higher specificity, offering more concrete steps or external resources for developers to address cryptographic misuses. However, no LLM was universally superior across both metrics. This highlights an area for improvement in LLM-based tools, as security professionals and developers rely heavily on detailed and actionable guidance to quickly implement fixes.

The high potential of LLMs in security analysis cannot be overlooked. However, we acknowledge that their current limitations need to be addressed before they can effectively replace or complement existing security tools in a real-world setting. To improve their performance, it will be critical to reduce false positives, fine-tune the models to understand cryptographic vulnerabilities better, and enhance their specificity by providing more detailed and practical guidance. Specialized training, focusing on cryptographic

misuse detection, could also help LLMs become more reliable in this domain. The development of fine-tuned LLMs tailored to specific security needs could help mitigate some of the challenges we encountered in detecting cryptographic misuse using LLMs.

Furthermore, the integration of LLMs into continuous integration/continuous deployment (CI/CD) pipelines or as part of a developer's integrated development environment (IDE) presents a promising application. However, their effectiveness in such workflows will depend on their ability to provide timely and accurate advice without overwhelming developers with false positives. Fine-tuning models to work within the constraints of such environments, ensuring they interact seamlessly with existing tools like Snyk could facilitate smoother integration into the software development lifecycle.

In conclusion, while our study has shown that LLMs hold significant promise in the field of cryptographic vulnerability detection, there remain critical hurdles to overcome. Addressing the issue of false positives, improving the specificity of advice, and further tailoring models to handle cryptographic vulnerabilities will be key to the widespread adoption of these AI-driven tools. However, our findings also suggest that no single tool or LLM is universally superior across all use cases. The choice of tool or model depends on the specific requirements of the task at hand, as each has its strengths and limitations. As these models evolve, they could serve as valuable supplements to traditional security tools, potentially providing developers with more intelligent and context-aware security guidance. Future research should focus on refining these tools and exploring their integration with current security practices to enhance both the speed and accuracy of cryptographic vulnerability detection, ultimately enabling developers to write secure code.

Bibliography

[1] Sharmin Afrose, Sazzadur Rahaman, and Danfeng Yao. CryptoAPI-Bench: A Comprehensive Benchmark on Java Cryptographic API Misuses. In *2019 IEEE Cybersecurity Development (SecDev)*, pages 49–61, 2019.

[2] Amazon Q Developer. Amazon Q Developer. `https://aws.amazon.com/q/developer/`, n.d. Retrieved August 21, 2024.

[3] Amit Seal Ami, Nathan Cooper, Kaushal Kafle, Kevin Moran, Denys Poshyvanyk, and Adwait Nadkarni. Why Crypto-detectors Fail: A Systematic Evaluation of Cryptographic Misuse Detection Techniques. In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 614–631, 2022.

[4] ChatGPT. ChatGPT. `https://chat.openai.com/`, n.d. Retrieved August 21, 2024.

[5] Alexia Chatzikonstantinou, Christoforos Ntantogian, Georgios Karopoulos, and Christos Xenakis. Evaluation of Cryptography Usage in Android Applications. In *Proceedings of the 9th EAI International Conference on Bio-Inspired Information and Communications Technologies (Formerly BIONETICS)*, BICT'15, page 83–90, Brussels, BEL, 2016. ICST (Institute for Computer Sciences, Social-Informatics and Telecommunications Engineering).

[6] Y. Chen, Y. Liu, K. L. Wu, D. V. Le, and S. Y. Chau. Towards Precise Reporting of Cryptographic Misuses. In *Proceedings 2024 Network and Distributed System Security Symposium*, 2024.

[7] Claude. Claude. `https://claude.ai/`, n.d. Retrieved December 21, 2024.

[8] CryptoGuardOSS. cryptoguardoss/cryptoguard. `https://github.com/CryptoGuardOSS/cryptoguard`, 2020. [Online; accessed August 21, 2024].

[9] CVE. `https://cve.mitre.org/cgi-bin/cvename.cgi?name=cve-2014-0160`, n.d. Retrieved January 21, 2025.

[10] CWE. `https://cwe.mitre.org/data/definitions/320.html`, n.d. Retrieved January 21, 2025.

[11] CWE - Common Weakness Enumeration. CWE - Common Weakness Enumeration. `https://cwe.mitre.org`, 2021. [Online; accessed August 21, 2024].

[12] DeepSeek. DeepSeek. `https://www.deepseek.com/`, n.d. Retrieved February 1, 2025.

[13] Manuel Egele, David Brumley, Yanick Fratantonio, and Christopher Kruegel. An Empirical Study of Cryptographic Misuse in Android Applications. In *Proceedings of the 2013 ACM SIGSAC Conference on Computer & Communications Security*, CCS '13, page 73–84, New York, NY, USA, 2013. Association for Computing Machinery.

[14] Chongzhou Fang, Ning Miao, Shaurya Srivastav, Jialin Liu, Ruoyu Zhang, Ruijie Fang, Asmita, Ryan Tsang, Najmeh Nazari, Han Wang, and Houman Homayoun. Large Language Models for Code Analysis: Do LLMs Really Do Their Job? In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 829–846, Philadelphia, PA, August 2024. USENIX Association.

[15] Ehsan Firouzi, Mohammad Ghafari, and Mike Ebrahimi. ChatGPT's Potential in Cryptography Misuse Detection: A Comparative Analysis with Static Analysis Tools. In *Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, ESEM '24, page 582–588, New York, NY, USA, 2024. Association for Computing Machinery.

[16] Felix Fischer, Yannick Stachelscheid, and Jens Grossklags. The effect of google search on software security: Unobtrusive security interventions via content re-ranking. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, CCS '21, page 3070–3084, New York, NY, USA, 2021. Association for Computing Machinery.

[17] Felix Fischer, Huang Xiao, Ching-Yu Kao, Yannick Stachelscheid, Benjamin Johnson, Danial Razar, Paul Fawkesley, Nat Buckley, Konstantin Böttinger, Paul Muntean, and Jens Grossklags. Stack overflow considered helpful! deep learning security nudges towards stronger cryptography. In *28th USENIX Security Symposium (USENIX Security 19)*, pages 339–356, Santa Clara, CA, August 2019. USENIX Association.

[18] Gemini. Gemini. `https://gemini.google.com/`, n.d. Retrieved August 21, 2024.

[19] GitHub AI Pair Programmer. GitHub AI Pair Programmer. `https://copilot.github.com`, n.d. Retrieved August 26, 2024.

[20] Mark G. Graff and Kenneth R. Van Wyk. *Secure Coding: Principles and Practices*. O'Reilly & Associates, Inc., USA, 2003.

[21] ISC2. `https://community.isc2.org/t5/Cloud-Security/Azure-Key-Vault-Vulnerabilities-Could-Leak-Sensitive-Data-After/td-p/76679`, n.d. Retrieved January 21, 2025.

[22] Simon Kafader and Mohammad Ghafari. Fluentcrypto: Cryptography in easy mode. In *2021 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 402–412, 2021.

[23] Stefan Krüger, Sarah Nadi, Michael Reif, Karim Ali, Mira Mezini, Eric Bodden, Florian Göpfert, Felix Günther, Christian Weinert, Daniel Demmler, and Ram Kamath. CogniCrypt: Supporting developers in using cryptography. In *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 931–936, 2017.

[24] Stefan Krüger, Johannes Späth, Karim Ali, Eric Bodden, and Mira Mezini. CrySL: An Extensible Approach to Validating the Correct Usage of Cryptographic APIs. *IEEE Transactions on Software Engineering*, 47(11):2382–2400, 2021.

[25] David Lazar, Haogang Chen, Xi Wang, and Nickolai Zeldovich. Why does cryptographic software fail? A case study and open problems. In *Proceedings of 5th Asia-Pacific Workshop on Systems*, APSys '14, New York, NY, USA, 2014. Association for Computing Machinery.

[26] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d'Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. Competition-level code generation with AlphaCode. *Science*, 378(6624):1092–1097, December 2022.

[27] Peiyu Liu, Junming Liu, Lirong Fu, Kangjie Lu, Yifan Xia, Xuhong Zhang, Wenzhi Chen, Haiqin Weng, Shouling Ji, and Wenhai Wang. Exploring ChatGPT's Capabil-

ities on Vulnerability Management. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 811–828, Philadelphia, PA, August 2024. USENIX Association.

[28] Llama. Llama. `https://www.llama.com/`, n.d. Retrieved September 21, 2024.

[29] Zohaib Masood and Miguel Vargas Martin. Beyond static tools: Evaluating large language models for cryptographic misuse detection, 2024.

[30] Microsoft. `https://azure.microsoft.com/en-us/products/key-vaultl`, n.d. Retrieved January 21, 2025.

[31] Sarah Nadi, Stefan Krüger, Mira Mezini, and Eric Bodden. Jumping through hoops: why do Java developers struggle with cryptography APIs? In *Proceedings of the 38th International Conference on Software Engineering*, ICSE '16, page 935–946, New York, NY, USA, 2016. Association for Computing Machinery.

[32] Eng Lieh Ouh, Benjamin Kok Siew Gan, Kyong Jin Shim, and Swavek Wlodkowski. ChatGPT, Can You Generate Solutions for My Coding Exercises? An Evaluation on Its Effectiveness in an Undergraduate Java Programming Course. In *Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education V. 1*, ITiCSE 2023, pages 54–60, New York, NY, USA, 2023. Association for Computing Machinery.

[33] OWASP. `https://owasp.org/www-project-top-ten/`, n.d. Retrieved January 21, 2025.

[34] OWASP Benchmark. OWASP Benchmark. `https://owasp.org/www-project-benchmark/`. Accessed May, 2024.

[35] Luca Piccolboni, Giuseppe Di Guglielmo, Luca P. Carloni, and Simha Sethumadhavan. Crylogger: Detecting crypto misuses dynamically. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 1972–1989, 2021.

[36] QwenLM. QwenLM. `https://qwenlm.ai/`, n.d. Retrieved December 21, 2024.

[37] Sazzadur Rahaman, Ya Xiao, Sharmin Afrose, Fahad Shaon, Ke Tian, Miles Frantz, Murat Kantarcioglu, and Danfeng (Daphne) Yao. CryptoGuard: High Precision Detection of Cryptographic Vulnerabilities in Massive-sized Java Projects. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, CCS '19, page 2455–2472, New York, NY, USA, 2019. Association for Computing Machinery.

[38] Elissa M. Redmiles, Noel Warford, Amritha Jayanti, Aravind Koneru, Sean Kross, Miraida Morales, Rock Stevens, and Michelle L. Mazurek. A Comprehensive Quality Evaluation of Security and Privacy Advice on the Web. In *29th USENIX Security Symposium (USENIX Security 20)*, pages 89–108. USENIX Association, August 2020.

[39] Xiaoxue Ren, Xinyuan Ye, Zhenchang Xing, Xin Xia, Xiwei Xu, Liming Zhu, and Jianling Sun. Api-misuse detection driven by fine-grained api-constraint knowledge graph. In *2020 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 461–472, 2020.

[40] Ricardo Rodriguez Garcia, Julie Thorpe, and Miguel Vargas Martin. Crypto-assistant: Towards facilitating developer's encryption of sensitive data. In *2014 Twelfth Annual International Conference on Privacy, Security and Trust*, pages 342–346, 2014.

[41] E. Rostami and F. Karlsson. Qualitative Content Analysis of Actionable Advice in Information Security Policies – Introducing the Keyword Loss of Specificity Metric. *Information & Computer Security*, 32(4):492–508, 2024.

[42] Snyk. Snyk Code — Code Security Analysis and Fixes - Developer First SAST. `https://snyk.io/product/snyk-code/`, n.d. Retrieved August 20, 2024.

[43] Soot. Soot. `https://github.com/soot-oss/soot`, 2020. Accessed: August 26, 2024.

[44] Tabnine. Tabnine. `https://www.tabnine.com`, n.d. Retrieved August 21, 2024.

[45] Raja Vallée-Rai, Phong Co, Etienne Gagnon, Laurie Hendren, Patrick Lam, and Vijay Sundaresan. Soot - a Java bytecode optimization framework. In *Proceedings of the 1999 Conference of the Centre for Advanced Studies on Collaborative Research*, CASCON '99, page 13. IBM Press, 1999.

[46] Alma Whitten. *Making Security Usable*. PhD thesis, Carnegie Mellon University, 5000 Forbes Avenue, Pittsburgh, PA, USA, 2004.

[47] Yifan Xia, Zichen Xie, Peiyu Liu, Kangjie Lu, Yan Liu, Wenhai Wang, and Shouling Ji. Exploring Automatic Cryptographic API Misuse Detection in the Era of LLMs, 2024.

[48] Jing Xie, H. R. Lipford, and Bill Chu. Why do programmers make security errors? . In *2011 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC 2011)*, pages 161–164, Los Alamitos, CA, USA, September 2011. IEEE Computer Society.

[49] Li Zhang, Jiongyi Chen, Wenrui Diao, Shanqing Guo, Jian Weng, and Kehuan Zhang. CryptoREX: Large-scale Analysis of Cryptographic Misuse in IoT Devices. In *22nd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2019)*, pages 151–164, Chaoyang District, Beijing, September 2019. USENIX Association.

[50] Ying Zhang, Md Mahir Asef Kabir, Ya Xiao, Danfeng Yao, and Na Meng. Automatic Detection of Java Cryptographic API Misuses: Are We There Yet? *IEEE Transactions on Software Engineering*, 49(1):288–303, 2023.

Appendix A

Appendix

A.1 LLM Prompt

Large Language Models (LLMs) received standardized prompts for each test case from the three benchmarks to ensure their responses were consistent. The specific prompt used for each test case is shown in Listing A.11. During the experiments, we kept the default settings for model hyper-parameters like temperature, Top P, and frequency penalty to maintain each model's natural response style. Responses from GPT 4-o-mini were collected automatically through an API, while responses from Llama, Claude, and Gemini were gathered manually from its chat interface.

```
1 I want you to detect "Cryptographic misuses" in the given
     Java code by considering the cryptographic misuse
    definitions below.
2
3 Cryptographic misuses are deviations from best practices
    while incorporating cryptographic algorithms into your
     software that could potentially be exploited by an
    adversary. Broadly, cryptographic misuses can be
```

```
    referred to as bad programming practices that create
    vulnerabilities and are associated with design flaws
    and unsafe architectural choices.
4
5 Identify the following for each cryptographic misuse and
    include it in your response. Follow the same template
    for reporting multiple misuses.
6
7 1- Include a label as YES/NO if there are specifically
    cryptographic misuses in the given code or not.
8 2- Include the name of the method containing the misuse.
9 3- Specify a message highlighting the type of misuses.
    Provide a description in the explanation as to why it
    is highlighted as a misuse.
10 4- State the line from the code containing the misuses.
    Specifically mention the starting line number that is
    causing misuse.
11
12 Code:
13 // Code for Each test case is placed here
```

Listing A.11: Prompt Used to Query LLMs

A.2 LLM Responses

The responses from the LLMs were fairly similar in format due to the structured prompt, though some variations were observed. For instance, in certain cases, the explanation for misuse from GPT 4-o-mini appeared under tags such as "Misuse Description", while in others, it was labeled as "Explanation" or a similar term. An example of this can be seen in Listing A.23, which shows the response for the CryptoAPI Bench test case listed in Listing A.22. The misuse in this instance involves using the DES algorithm and the "DES" string to initialize the key.

```
1  package org.cryptoapi.bench.brokencrypto;
2
3  import java.security.InvalidKeyException;
4  import java.security.NoSuchAlgorithmException;
5  import javax.crypto.NoSuchPaddingException;
6
7  public class BrokenCryptoABMCCase1
8  {
9      public static void main(String[] args) throws
           NoSuchPaddingException, NoSuchAlgorithmException,
           InvalidKeyException
10     {
11         BrokenCryptoABMC1 bc = new BrokenCryptoABMC1();
12         String crypto = "DES/ECB/PKCS5Padding";
13         String cryptokey = "DES";
14         bc.go(crypto, cryptokey);
15     }
16 }
```

Listing A.22: CryptoAPI Bench Test Case: BrokenCryptoABMCCase1.java

```
Response:
1- **Label**: YES
2- **Method containing the misuse**: main
3- **Message highlighting the type of misuses**: Use of
   weak cryptographic algorithm and mode of operation
   **Explanation**: The code uses DES (Data Encryption
      Standard) in ECB (Electronic Codebook) mode, which
      is considered weak and insecure. DES itself has
      been deprecated due to its small key size and
      susceptibility to brute-force attacks, while ECB
      mode exposes patterns in plaintext since identical
      plaintext blocks produce identical ciphertext
      blocks. This can lead to serious vulnerabilities,
      allowing attackers to perform cryptanalysis or
      deduce information about the plaintext. Best
      practices recommend using stronger algorithms (e.g
      ., AES) and secure modes of operation (e.g., GCM or
       CBC with proper IV management).
4- **Line from the code containing the misuses**: 10-11 (
   Lines mentioning ‘String crypto = "DES/ECB/
   PKCS5Padding";‘ and ‘String cryptokey = "DES";‘)
```

Listing A.23: GPT 4-o-mini Response for BrokenCryptoABMCCase1.java

The results generated by the LLMs across all three benchmarks were compiled and compared with the dataset labels to draw our conclusions. A selection of LLM responses for various misuse instances is presented in Table A.21. The detailed findings are dis-

cussed in Chapter 4.

Table A.21: LLM Responses to Test Cases from Different Datasets

Dataset	Java File	Misuse Instance	Dataset Label	LLM	LLM Label	Message Highlighted	Message Description
CryptoAPI Bench	BrokenCryptoABICase13	String crypto = “RC4”;	TRUE	GPT	YES	Use of insecure cryptographic algorithm	The code uses “RC4” as the cryptographic algorithm. RC4 is considered insecure due to vulnerabilities that allow for various attacks, including key recovery and plaintext recovery. Best practices recommend avoiding its use in favor of more secure algorithms like AES. By allowing the specification of cryptographic algorithms at runtime, this code opens the door for an adversary to potentially use insecure algorithms deliberately.
CryptoAPI Bench	StaticSaltsABSCase1	byte[] salt = { -94 };	TRUE	Gemini	YES	Hardcoded Salt	The code contains a hardcoded salt value, which is a common cryptographic misuse. Hardcoded salts are problematic because they can weaken the security of the cryptographic system. If an attacker discovers the hardcoded salt, they can potentially compromise the security of the system. To address this issue, the salt value should be generated randomly and securely. This can be done using a cryptographically secure random number generator (CSPRNG). Additionally, the salt should be stored securely and not exposed to unauthorized access.
CryptoAPI Bench	BrokenCryptoBBCase5	Cipher cipher = Cipher.getInstance(“IDEA”);	TRUE	GPT	YES	Use of weak cryptographic algorithm (IDEA) and key size analysis	The method utilizes the IDEA (International Data Encryption Algorithm), which is considered outdated and vulnerable due to its small block size and usage of relatively small key lengths (up to 128 bits). Modern best practices recommend using stronger algorithms such as AES (Advanced Encryption Standard), which offers better security and is more widely supported.

CryptoAPI Bench	BrokenHashABPSCase3	md = MessageDigest.getInstance("SHA-256");	FALSE	Gemini	NO	N/A	This code snippet does not contain any specific cryptographic misuses based on the provided definition.
MASC	AES String Replace	Cipher c = Cipher.getInstance("AES".replace("A", "D"));	TRUE	GPT	YES	Misuse of unapproved algorithm specification for Cipher	The usage of ``AES".replace(``A", ``D") is a significant misuse. This expression incorrectly modifies the string that is supposed to define the algorithm name, changing it from "AES" to "DES". Since "DES" is a less secure algorithm, this can lead to vulnerabilities in the encryption implementation realized by this code. This does not represent a proper way to specify or select cryptographic algorithms, as it can easily be manipulated or misunderstood.
MASC	String Case Transform	c = Cipher.getInstance("des".toUpperCase(Locale.ENG));	TRUE	Gemini	FALSE	This code snippet does not contain any specific cryptographic misuses.	N/A
MASC	Static IV	byte[] bytes = "Hello".getBytes();	TRUE	GPT	YES	Use of a static IV (Initialization Vector)	The code creates an `IvParameterSpec` using a static byte array derived from the string "Hello". Using a static IV is a significant cryptographic misuse because IVs are supposed to introduce randomness to ensure that the same plaintext encrypted multiple times will yield different ciphertexts. This can lead to vulnerabilities such as revealing patterns in the plaintext and makes the encryption susceptible to certain attacks. A random or unique IV should be generated each time encryption is performed.

MASC	Replace SHA256 to DES	cryptoDigest = MessageDigest.getInstance("SHA-256".replace("SHA-256", "MD5"));	TRUE	Gemini	YES	Using a String manipulation technique to alter the algorithm name is a misuse.	The code attempts to dynamically change the MessageDigest algorithm from SHA-256 to MD5 using string manipulation. This is a cryptographic misuse because:Insecure Algorithm Selection: Hardcoding the algorithm name within the code makes it difficult to manage and update. It also increases the risk of accidentally using a weaker algorithm (MD5 in this case) compared to the intended stronger one (SHA-256).
OWASP	BenchmarkTest00070.java	MessageDigest md = MessageDigest.getInstance("SHA1", "SUN");	TRUE	GPT	YES	The code uses SHA1 for hashing sensitive data	SHA1 is considered cryptographically broken and unsuitable for further use. It is vulnerable to collision attacks, where two different inputs can result in the same hash output. In situations where security is critical (like storing hashes of passwords or sensitive information), a more robust hashing function such as SHA-256 or better yet, a key derivation function like PBKDF2, bcrypt, or Argon2 should be employed instead.
OWASP	BenchmarkTest02291.java	Cipher c = Cipher.getInstance("DES/CBC/PKCS5Padding", "SunJCE");	TRUE	Gemini	YES	Weak Encryption Algorithm and Key Size	The code uses the DES/CBC/PKCS5Padding algorithm for encryption. DES is a legacy algorithm with a small key size (56 bits) and is considered insecure by modern standards. It's vulnerable to brute-force attacks. Additionally, the key is generated using KeyGenerator.getInstance("DES") which reinforces the use of a weak key size.

OWASP	BenchmarkTest00370.java	md = MessageDigest.getInstance("sha-384", "SUN");	FALSE	GPT	YES	Using a weak hashing algorithm (SHA-384) to hash sensitive data.	While SHA-384 is a part of the SHA-2 family of cryptographic functions, using it alone for hashing passwords is considered a misuse because it is not a key derivation function and lacks sufficient resistance against brute-force attacks. Best practices recommend using algorithms specifically designed for hashing passwords, such as bcrypt, scrypt, or Argon2, which incorporate salting and an adjustable number of iterations to slow down attackers.
OWASP	BenchmarkTest00270.java	MessageDigest md = MessageDigest.getInstance("SHA-512", "SUN");	FALSE	Gemini	NO	N/A	There are no cryptographic misuses detected in the code.
MASC	StringReplace.java	("MD5".*replace*("", ""));	TRUE	LLAMA	YES	The code uses a weak and broken hashing algorithm	MD5 is a widely used but cryptographically broken hash function. It is vulnerable to collisions, meaning it's possible to find two different input strings that produce the same output hash. This can lead to various security issues, such as the ability to forge digital signatures or data integrity compromises.
MASC	StringReplace.java	("MD$5".replace("$", ""));	TRUE	LLAMA	YES	The code uses a weak and broken hashing algorithm	MD5 is a widely used but cryptographically broken hash function. It is vulnerable to collisions, meaning it's possible to find two different input strings that produce the same output hash. This can lead to various security issues, such as the ability to forge digital signatures or data integrity compromises.
MASC	differentCase.java	getInstance("md5");	TRUE	LLAMA	YES	Insecure Hashing Algorithm Used	The code uses the MD5 hashing algorithm, which is considered insecure for cryptographic purposes. MD5 has been shown to be vulnerable to collisions and is no longer recommended for use in secure applications. Recommendation: Use a more secure hashing algorithm like SHA-256 or SHA-3 instead of MD5.